SPEC · OMEN · SUPERSEDED FOR CRM SYNC

Omen — Functional Specification & UAT Release Plan

The Omen-build functional specification and UAT release plan — the engineering record preceding the CRM Sync product spec.

Document ID: **CRM-FUNC-SPEC-001** Version: **1.19** Date: **2026-06-17**

Status: **Draft – Pending DPO & PMO Review** Classification: **Public**

> **Superseded for CRM Sync (2026-07-12):** This document is the engineering record of the **Omen** build (the `omenphase1-1.webflow.io` era) and is preserved as-is, including its original pricing model. The canonical specification for the **CRM Sync** product is CRM Sync — Functional Specification.

Document Control

ROLE	NAME	SIGNATURE	DATE
DPO (Data Protection Officer)	_____	_____	_____
PMO (Project Management Office)	_____	_____	_____
Engineering Lead	_____	_____	_____
QA Lead	_____	_____	_____

Revision History

VERSION	DATE	AUTHOR	CHANGES
1.0	2026-05-14	Engineering	Initial functional spec with UAT test plan
1.1	2026-05-14	Engineering	Security hardening: HMAC verification on all webhooks/GDPR handlers, OAuth state + shop domain validation, POST /config authentication, secrets stripped from embed HTML, PII console.log removed, compliance webhook dispatcher, welcome email flow, Webflow OAuth
1.2	2026-05-17	Engineering	Multi-tenant SaaS architecture (per-shop KV isolation), ADMIN_KEY bearer token auth on all admin/write endpoints, Cloudflare Access (Zero Trust) email whitelisting, Adobe Experience Platform streaming integration with SHA-256 PII hashing, three-tier pricing model (Shared/Private/Enterprise), Webflow extension Plan tab and Adobe config UI
1.3	2026-05-18	Engineering	Region-based tenant groups (US/CA/DE/FR/UK), structured tenant registry (`TenantEntry[]` with region + timestamp), per-tenant admin keys with platform key fallback, UUID-keyed OAuth state (no more global singleton), `tenantKvKey()` helper for all KV operations, `POST /admin/provision-region` with Xano schema replication, `GET/POST /platform/config` for shared credentials, extension `?shop=` on all fetch calls, multi-tenant isolation test suite
1.4	2026-05-19	Engineering	UCP/A2A/AP2 protocol endpoints (14 new routes, 81 total), Storefront Cart API checkout with channel attribution, A2A/AP2 consent toggles, embed feature flags (`embeds_enabled`), GA4 UCP events (`ucp_checkout_created` , `ucp_mandate_created` , `ucp_mandate_used` , `ucp_checkout_completed`), auto-provisioned Storefront Access Token, cart embed, Protocol Status UI in account/dashboard embeds, AP2 mandate HMAC signatures
1.5	2026-05-20	Engineering	Shopify expiring token rotation (mandatory since April 2026), `POST /admin/shopify-refresh` manual refresh endpoint (82 routes), OAuth scopes expanded (`read_products` , `read_orders`), cart embed multi-tenant `SHOP_PARAM` injection, UCP Entitlement (A2A/AP2) toggles in extension Auth tab, scope management via `shopify.app.crm-sync.toml` , webhook API version `2026-04`
1.6	2026-05-20	Engineering	Token Lifecycle Management specification (§3.14): Shopify expiring token architecture, refresh flow, multi-tenant token storage schema, three-surface token sync (app loader, cron, manual), diagnostic endpoints, embed system updated to fetch+innerHTML pattern with script re-execution

VERSION	DATE	AUTHOR	CHANGES
1.7	2026-05-21	Engineering	Removed `/embed/cart` route and Product Cart embed — storefront and cart handled by Shopify Web Components + PIM grid (`cf-worker-webflow-sync`), reducing worker to ~9,400 lines / 78 routes; client credentials grant for own-store token provisioning; `sanitizeDomain()` applied at all ingress points; domain trailing-comma fix; Shopify Dev Dashboard terminology alignment (removed all `shpat_` / `shpua_` / `shprt_` prefix assumptions)
1.8	2026-05-26	Engineering	Self-service admin key rotation (§3.10.2, FR-PRICING-07): two-tier key hierarchy (root + rotatable), `POST/GET/DELETE /admin/rotate-key` endpoints, Persona C multi-tenant Shopify key management independent of Shopify OAuth install; rotatable key accepted across all admin auth surfaces (`verifyBearerToken` , `verifyAdminKey` , `verifyBearerOrTenantToken`)
1.9	2026-05-28	Engineering	Shopify embedded admin app dashboard fixed (§3.15, FR-APP): URL-driven tab navigation (`?tab=Config|Embeds|Settings`) replacing client-only `useState` , so tabs switch via real navigation and work even before/without client hydration; native Polaris `s-page` + `s-button` tab strip; Config/Embeds/Settings consolidated into the single `/app` route (removed standalone `app.embeds` / `app.settings` routes and `s-app-nav`); save form hardened with hidden `intent` field for pre-hydration POST
1.10	2026-05-28	Engineering	Security: fixed fail-open auth on AP2 mandate reads (`handleMandateGet`) — it passed a synthetic `{ADMIN_KEY: cfg.adminKey}` env to `verifyBearerToken` , tripping the "no keys configured = open" dev shortcut when the per-tenant key was empty, so unauthenticated reads leaked `404` (mandate existence) instead of `401` . Now passes the real `env` + tenant + rotatable keys (fail-closed). Smoke test repointed to the CRM worker (`cf-worker-crm-sync`) with `GET /config` →401 and `/setup` →302 (Cloudflare Access) expectations, stale `/embed/cart` checks removed, and a new **Mandates & Permissions** section (UAT-SEC-21..23)
1.15	2026-06-12	Engineering	**Agency → Client Deploy Handoff + Interactive Key Rotation (§13, FR-HANDOFF-01..05):** normative handoff checklist (public `AGENCY-HANDOFF.md`): re-mint-never-transfer credential transfer in dependency order, Interactive Key Rotation ceremony (Security Human executes, Agent verifies + documents; fingerprint-only audit records), rotation triggers (handoff/90d/personnel/incident/scope), Legal/Compliance-PII package (PII map, processor chain, access matrix incl. agents, custody evidence), quarterly Security-Scaling Report

VERSION	DATE	AUTHOR	CHANGES
1.13	2026-06-02	Engineering	**AI/API DevOps App Harness model + gating simplification + versioning (§2.0, §5.0, §12):** documented the two-plane harness (Cloudflare Workers compile/integration ⇄ Xano K8s/Docker/Redis data; both Shopify + Webflow apps compile on Cloudflare; CORS-restricted domains, CI/CD, edge scaling; Data Channels/API-AI/Skills/Schema/JSON Feeds). **Gating simplified:** `ADMIN_KEY` is the canonical gateway (free self-setup); `SHOPIFY_APP_SECRET` **dropped as an admin login** (HMAC-only) and the Shopify app migrated to `ADMIN_KEY`; `keyEq` trims whitespace; fail-open is local-dev-only. **Custom-DNS upsell** (shared `story-story.ai` → own namespace e.g. `ysl150.com`). Added the **Versioning Model** (§12).
1.12	2026-06-02	Engineering	**Multi-tenant scaling conventions + Stakeholder Forward-Deploy Harness (§3.17, FR-ENV/FR-LOC/FR-FDH):** three deploy channels (Dev/Stage/Prod) with per-hostname env/deploy-team labels (editable in portal; ownable by the Webflow Publish Refactor); localization extended to a geo model (NA/EMEA/APAC/LATAM) — markets US/CA/UK/DE/FR/AU/NZ/SG/JP/MX/BR with locale+currency, prefix/suffix shop naming + country-TLD inference, entitlement currency defaulted per market; deploy→state binding surfaced (Webflow UI + Xano Data) in the portal; `/setup` Agentic Commerce readiness + portal env/localization banners; `GET /env-label`; Webflow extension shows env·team chip + Config Portal link
1.14	2026-06-07	Engineering	**Shopify token grant types & self-heal recovery (§3.14.6, FR-TOKEN-11..13):** documented both supported grants — Refresh Token (merchant OAuth) and Client Credentials (own-store, no refresh token) — and automatic selection. Added the **dead-refresh-token self-heal**: a refresh grant that returns `400 "requires an active refresh_token"` now clears the dead token and falls through to Client Credentials, recovering without an OAuth re-install (prior symptom: all Shopify-backed tools — product search, cart, checkout, orders — silently return empty). Added on-`401` mid-request refresh+retry so a token failing before its recorded expiry self-heals, plus a failure-reference table.

VERSION	DATE	AUTHOR	CHANGES
1.11	2026-06-02	Engineering	**Agentic Commerce — Data Entitlement, Consent Engine & Enterprise Add-ons (§3.16, FR-ENT):** Xano-authoritative entitlements (tables 190–194) as source of truth for tenant/user/agent permissions + consent; EdDSA (Ed25519) offline-verifiable tokens with non-destructive `next→active→retired` signing-key rotation; scoped `entitlement:read` key (`XANO_ENTITLEMENT_KEY`) on the read path (never the master meta key); omni-channel poll-on-change projection (Cloudflare/Shopify/GA4 real-time + Adobe/SAP/Nielsen batch) with **version-monotonic consent resolution** (per- `(channel,subject)` `state_version` guard → consent never regresses across mixed cadences) + cursor replay; **first-class versioned consent** (GA4 Consent Mode v2) carried in snapshot + token; enterprise add-ons gated by `entitlement.features[]` with per-Org connectors; `/settings` operations console; Clean Room re-scoped as a downstream consumer of versioned consent (§6.4)
1.16	2026-06-13	Engineering	**DevOps modular build strategy + dynamic data "loops" (§2.5, §3.18, FR-BUILD-01..06):** documented the independently-deployable module model (CRM Sync / PIM Sync / Webflow extension / Shopify app / public docs — each its own worker, config, and KV; no synchronous cross-module calls; a storefront runs PIM-only or CRM-only, CRM embeds gated server-side by `embeds_enabled` returning an **empty 200** + `X-CRM-Embed-Disabled` , never an error body) and the **dynamic data loops** that keep resources fresh without blocking reads or coupling modules (export→ingest→publish headless rail, event-driven `content-drain` , stale-while-revalidate + in-isolate single-flight read caches, token self-heal, version-monotonic consent projection). PDP perf: `/product` + shop-catalog SWR/single-flight (cold ~1.8 s×3 → ~240 ms). Header/footer non-destructive rules + enforcing harness suite.
1.18	2026-06-15	Engineering	**Interactive Key Ceremony promoted to a primary feature (§1.1, §13 FR-HANDOFF-02; `KEY-MANAGEMENT-LIFECYCLE.md` §9):** agent-safe credential operations documented end-to-end — every privileged key mint/rotate/revoke/set runs as a two-role ceremony where a named Security Human *executes* (interactively) and an Agent *prepares/verifies/records* but never mints, holds, or transports a secret. Safe-by-construction: permission-classifier blocks agent-side mints (handed to the operator, never worked around), secret values written straight to `chmod 600` files / piped into the secrets manager (read-back masked to `(set)` /preview), additive-only rotation (`admin_key:rotatable` alongside an immovable root). Added the ceremony flow + trust-boundary diagrams and the fingerprint-only (`sha256[:8]`) audit-record schema; featured in `SECURITY-POSTURE.md` §"Agent-Safe Credential Operations".

VERSION	DATE	AUTHOR	CHANGES
1.19	2026-06-17	Engineering	**Cost Architecture — Token-Maxing Resolution (§3.20, FR-COST-01..06):** documented the cost model that resolves agentic commerce's "token maxing" (runaway LLM-token + per-request/egress spend). Data Resolution (joins/reconciliation/entity serving) + encryption/key logic run on the **fixed-fee, no-egress Xano plane** (§2.0's Xano K8s/Docker/Redis data plane); the cache-fed agent loop (Anthropic *Tool Runner* as a read-only consumer) stays thin and never re-derives data by re-prompting. Unbounded-volume work lands on a flat cost floor while token spend stays bounded — the economic basis for the flat FR-PRICING (§3.10) tiers, pairing cost predictability with the security boundary (keys/crypto never egress; §3.16, §13).
1.17	2026-06-13	Engineering	**Chatbot FAQ answer cascade (§3.19, FR-FAQ-01..06):** documented the tiered `/commerce/faq` cascade (Shopify KB → guide-list → weighted Xano FAQ → blog-intent → locale-filtered vectorized KB → Botpress KB → blog last-resort). The Botpress KB — which merges the multilingual product PDFs with the FAQs and matches cross-lingually — is now a **locale-gated last-resort fallback**, not the primary source: as primary it surfaced foreign-language (e.g. Vietnamese) manual chunks and garbled title matches that preempted clean answers. Added `passageLocaleOk()` script/ASCII gating + a prose-length floor, and the guide-list intent that returns the full PDF catalog. `BOTPRESS_PAT`-gated (absent → structured cascade runs alone).
1.20	2026-06-21	Engineering	**Globalized Commerce — Agentic Settlement Layer + Glossary (Payments surface):** added the market/rail/settlement-socket/mandate layer surfaced on the **Payments** operator screen (`/embed/payments` ; tabs Rails·Sockets·Mandates·History). Two-tier market model — Tier-1 Stripe-Funnel **ratified** (JP/KR) vs Tier-2 Guarded-Modular **incubating/blocked** — with a fail-closed runtime guardrail (`assertMarketActive`) **and** a pre-deploy guard, so a non-ratified market cannot reach active settlement dispatch. Pluggable per-Country/Bank **settlement sockets** (Stripe / Kakao / domestic PG, health-gated) extend the socket-links pattern to money; Merchant-of-Record split (`platform_local` / `platform_xborder`); JWE-+-Auth-Me credential binding before capture. `GET /commerce/markets` now stamps each market's real lifecycle; new `GET /commerce/settlement/sockets` . Added the **Glossary — Globalized Commerce** (markets/rails/lifecycle, sockets/MoR, mandates, settlement state; the two-"ratify" distinction).

1. Executive Summary

CRM Sync is a multi-tenant server-side customer relationship management SaaS (~9,400 lines, single-file Cloudflare Worker, 78 routes) that synchronizes user identity, consent, segmentation, and campaign tags across seven integrated services. Product display and cart/checkout are handled externally by Shopify Web Components and a PIM grid worker (`cf-worker-webflow-sync`), keeping this worker focused on CRM, consent, and identity. The system operates with tri-directional data flow between Shopify (commerce), Xano (database), Webflow CMS (content), Google Analytics GA4 (measurement), Adobe Experience Platform (CDP), Resend (transactional email), and a Webflow-embedded frontend. It is a registered Shopify App and Webflow Marketplace App, subject to both platforms' submission and compliance requirements. The system supports multiple tenants (Shopify shops) with isolated KV-backed configuration, and is sold as a SaaS product with Shared ($69/mo), Private ($325/mo), and Enterprise (custom) pricing tiers.

1.1 Business Objectives

- Centralized consent management with auditable provenance
- Multi-tenant SaaS with per-shop configuration isolation
- Real-time CRM tag synchronization across all customer-facing channels
- GDPR/CCPA-compliant data handling with right-to-erasure and data portability
- Server-side analytics that respect user consent state
- Adobe Experience Platform streaming ingestion with SHA-256 hashed PII
- Self-service user control panel (UCP) for consent and preference management
- Tiered SaaS pricing with Shopify App Billing integration

- **Agent-safe credential operations** — privileged key mint/rotate/revoke runs as an Interactive Key Ceremony (Security Human executes, Agent verifies and records; secrets never enter logs, transcripts, or agent context), with a fingerprint-only audit trail (§13, FR-HANDOFF-02; `KEY-MANAGEMENT-LIFECYCLE.md` §9)

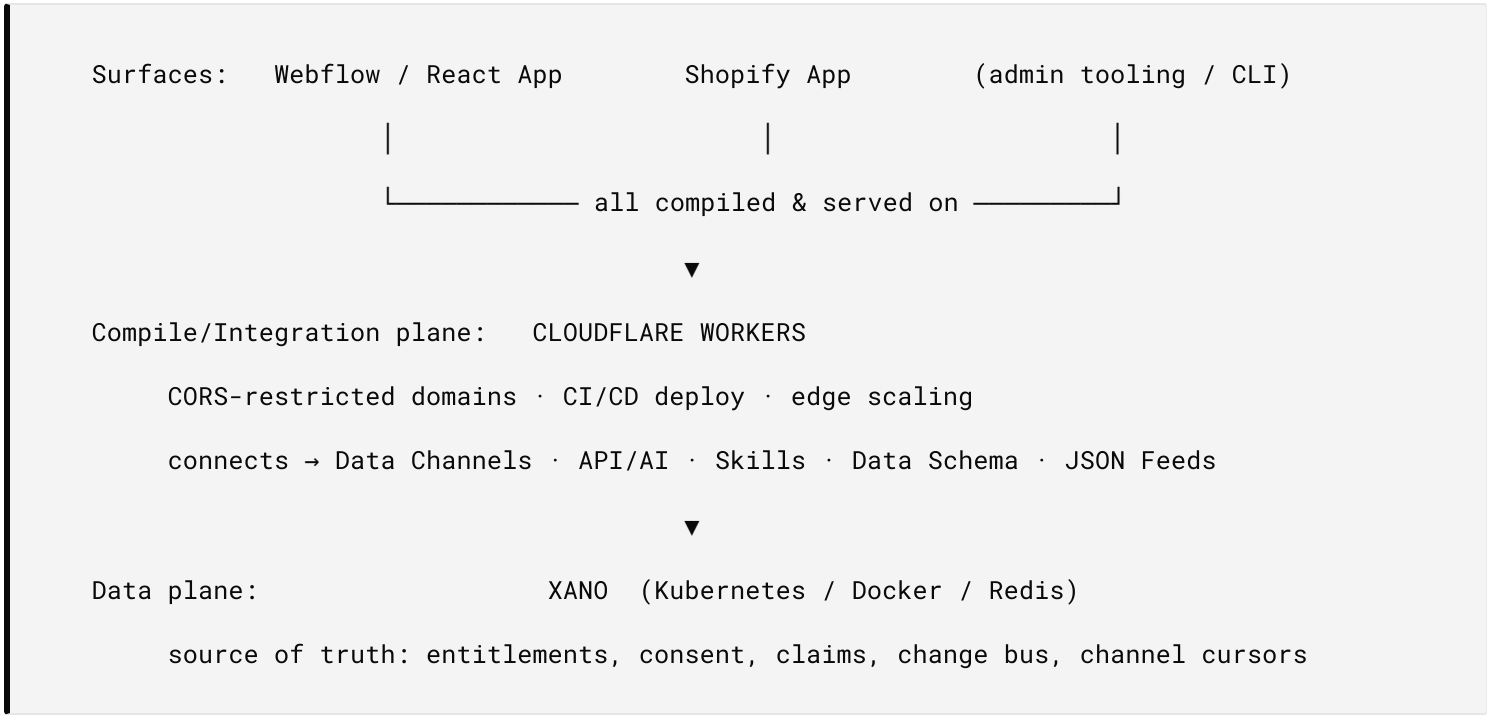
1.2 Deployment Environment

COMPONENT	PLATFORM	IDENTIFIER
Setup Wizard	Cloudflare Workers	`crm.story-story.ai/setup`
Backend Worker	Cloudflare Workers	`cf-worker-crm-sync`
Database	Xano	`xerb-qpd6-hd8t.n7.xano.io`
Commerce	Shopify Admin API	Per-tenant (e.g., `hx-stage.myshopify.com`)
CMS	Webflow CMS API v2	Per-tenant (e.g., `omenphase1-1.webflow.io`)
Analytics	GA4 Measurement Protocol	Per-tenant (e.g., `G-S7QGFWPZ8X`)
CDP	Adobe Experience Platform	Per-tenant AEP streaming via `dcs.adobedc.net`
Email	Resend API	`story-story.ai` domain
Config Store	Cloudflare KV	`CRM_STATE` (tenant-prefixed keys)
Access Control	Cloudflare Zero Trust	`kcoop.cloudflareaccess.com`

2. System Architecture

2.0 AI/API DevOps App Harness — System Model

Two planes, three surfaces, one deploy harness.



- **Cloudflare Workers = the compile/integration plane.** *Both* the Shopify app (`hx-crm-sync`, React Router) and the Webflow/React app (CRM Auth extension + `/settings` portal) **compile and run on Cloudflare** — one compute plane for both surfaces. The worker connects the five integration surfaces: **Data Channels** (channel adapters + change bus), **API/AI** (UCP A2A/AP2 + EdDSA offline tokens), **Skills** (entitlement `features[]`), **Data Schema** (Xano tables + Data-Funnel mappings/rules), **JSON Feeds** (`/channels`, `/.well-known/ucp`, manifests).
- **Xano = the data plane** (Kubernetes / Docker / Redis): the authoritative store for entitlements (190–194), consent, claims, the change bus, and channel cursors.
- **Domains are CORS-restricted, deployed via CI/CD, and scale at the edge.** The shared configuration URL is `story-story.ai`; a tenant can run the portal on its **own DNS namespace** (e.g. `ysl150.com`) as a **paid upsell** (Private/Enterprise). The gateway credential is `ADMIN_KEY` — anyone can stand up their **own** instance *free* by setting their own `ADMIN_KEY`; the shared/custom-DNS tiers are the monetized layers on top.

2.1 Multi-Tenant Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│  Cloudflare KV (CRM_STATE)                                       │
│                                                                  │
│  tenant:{shop-a}:config → CrmSiteConfig (Shop A)                 │
│  tenant:{shop-b}:config → CrmSiteConfig (Shop B)                 │
│  tenant:{shop}:tag_table_ids         → Xano table IDs            │
│  tenant:{shop}:webflow_tags_collection_id → Webflow collection ID │
│  tenant:{shop}:sync:customers:last_run    → ISO timestamp        │
│  tenants:index → TenantEntry[] (shop, region, registered_at)     │
│  platform:config → PlatformConfig (shared creds)                 │
│  oauth_state:{uuid} → { shop, return_to } (10min TTL)            │
└──────────────────────────────────────────────────────────────────┘
```

Region-Based Tenant Groups — tenants are organized by region prefix in their shop domain:

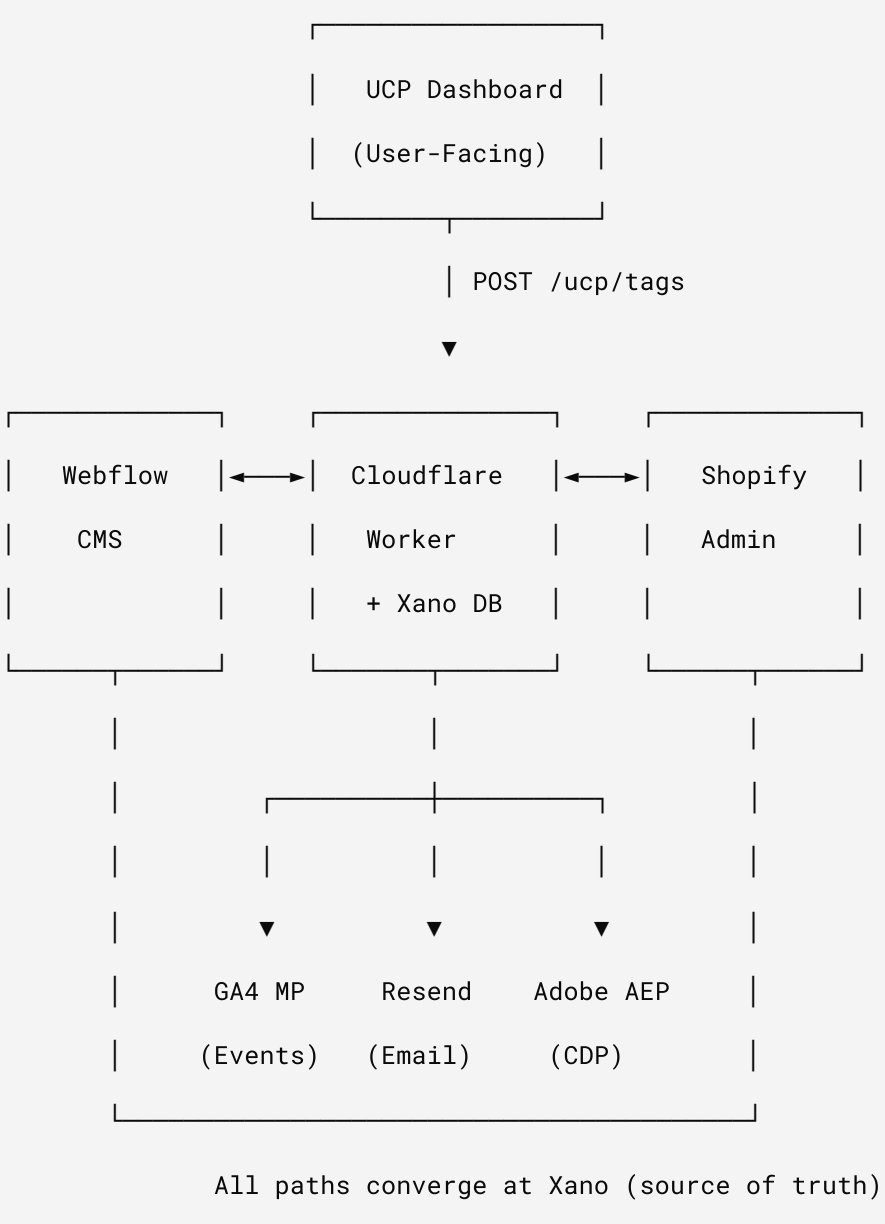
REGION	NAMING CONVENTION	EXAMPLE
US	`us-{brand}.myshopify.com`	`us-hx.myshopify.com`
CA	`ca-{brand}.myshopify.com`	`ca-hx.myshopify.com`
DE	`de-{brand}.myshopify.com`	`de-hx.myshopify.com`
FR	`fr-{brand}.myshopify.com`	`fr-hx.myshopify.com`
UK	`uk-{brand}.myshopify.com`	`uk-hx.myshopify.com`

Region is auto-inferred from the shop domain prefix via `inferRegionFromShop()`, or can be set explicitly via the config POST body or `POST /admin/provision-region`.

Tenant Identity Resolution — each ingress identifies the tenant differently:

INGRESS	TENANT ID SOURCE
Shopify embedded app	Session → `shop` domain (via `/config` POST)
Shopify webhooks	`X-Shopify-Shop-Domain` header
Storefront embeds	`?shop=` query param
Webflow extension	`?shop=` query param (configured during setup)
OAuth install flow	`?shop=` query param
Cron/scheduled	Iterates `tenants:index` registry
Worker `/settings`	`?shop=` query param (after admin auth)
Admin endpoints	`?shop=` query param + Bearer token

2.2 Tri-Directional Sync Model



2.3 Data Flow Directions

#	DIRECTION	TRIGGER	PATH
1	Shopify → Xano → Webflow → GA4	Cron (`*/15 * * * *`) or `POST /webhooks/customer-update`	Customer data pulled from Shopify, upserted to Xano, pushed to Webflow CMS + GA4 user properties
2	Webflow → Xano → Shopify → GA4	`POST /webhooks/webflow-item-changed` (auto-registered)	CMS item edited, fields synced to Xano, tags/metafields pushed to Shopify + GA4
3	UCP → Xano → Shopify + Webflow → GA4	`POST /ucp/tags` or `POST /tags/user`	User self-service tag/consent change flows through all channels
4	Form Bridge → UCP → All	Any `data-crm-form` form submission	Auto-tags, consent logging, GA4 event, full channel flow
5	Shopify → Xano → Resend (welcome)	`POST /webhooks/customer-create` or cron sync (new user)	New Shopify-origin user created in Xano, welcome email sent via Resend with set-password link
6	Shopify → Worker (compliance)	`POST /api/webhooks` (TOML `compliance_topics`)	Dispatcher routes by `X-Shopify-Topic` to GDPR handlers; all HMAC-verified
7	Shopify → Xano → Adobe AEP	`POST /webhooks/customer-update` (if `adobe_aep_enabled`)	Customer data hashed (SHA-256) and streamed to AEP dataset via XDM ExperienceEvent; sync status written to `user_extras`

2.4 Config Precedence

```
KV Store (tenant:{shop}:config) > wrangler.toml [vars] > Wrangler Secrets
```

For multi-tenant mode, config is resolved per-shop: `getTenantConfig(env, shop)` reads `tenant:{shop}:config` from KV, then merges with environment variable defaults. Legacy single-tenant mode falls back to the `crm_config` key.

2.5 DevOps Modular Build Strategy & Dynamic Data "Loops"

The platform is built as **independently deployable modules** wired together by **dynamic data "loops"** — closed feedback cycles that keep each module's data resources fresh *without* coupling the modules at request time. The build strategy gives the blast-radius isolation; the loops give the consistency.

Modular build strategy. Each surface is its own deployable unit — own build pipeline, own config, own storage — and none calls another synchronously on the request path:

MODULE	UNIT	DEPLOY	STORAGE	STANDALONE?
CRM Sync	`cf-worker-crm-sync`	`wrangler deploy` (own `wrangler.toml`)	KV `CRM_STATE`	✓ chat / auth / consent / KB
PIM Sync	`cf-worker-webflow-sync`	`wrangler deploy` (own `wrangler.toml`)	KV `WEBFLOW_SYNC_STATE`	✓ catalog / PDP / cart / search
Webflow extension	`crm-auth` (Designer extension)	extension publish	—	✓ (client of CRM Sync)
Shopify app	`hx-crm-sync` (React Router)	`npm run deploy:app`	—	✓ (admin dashboard)
Public docs	`crm-sync-setup`	static Pages	—	✓

- **No runtime cross-calls.** PIM Sync and CRM Sync are independent clients of the same backends (Xano workspace 4, Shopify) — verified both directions; neither fetches the other. The sole link is the outbound, failure-tolerant `content-drain` event (below).
- **Graceful standalone degradation.** A storefront can run PIM-only or CRM-only. CRM embeds are gated server-side by the `embeds_enabled` allowlist; a disabled embed returns an **empty** `200` + `X-CRM-Embed-Disabled` (never an error body), and

the storefront loaders are catch-guarded — so a missing module is a clean no-op, not a site-wide breakage.

- **Independent versioning & blast radius.** Each module deploys on its own cadence; separate `wrangler.toml`, separate KV, separate Version IDs — a deploy to one cannot break another.

Dynamic data "loops." Data resources stay current through recurring, self-correcting loops rather than blocking reads or manual refresh. Each loop is owned by exactly one module and is **non-blocking** (background revalidate, fire-and-forget event, or scheduled cron):

LOOP	TRIGGER	CYCLE	KEEPS FRESH
Export → ingest → publish (headless rail)	content/catalog change + fingerprint cron	Shopify → Xano → token-gated `/export` feed → `repository_dispatch` → static build	Headless market builds
Event-driven content-drain	`content.changed`	PIM Sync → `POST /events/content-drain` → CRM Sync metaobject / KB mirror	KB / AEO mirror
Stale-while-revalidate read cache	read miss / age > soft-TTL	serve cached (or stale) instantly → background refresh (`ctx.waitUntil`) + in-isolate single-flight	`/product`, shop catalog, `/embed/footer`
Token self-heal	`401` / dead refresh token	refresh grant → fall through to client-credentials → retry	Shopify Admin tokens
Consent projection	per-channel change	version-monotonic poll-on-change + cursor replay	Omni-channel consent state

The two halves pair like this: the **modular build** keeps failure domains separate, while the **loops** are the only glue between them — event-driven or cache-driven, and always non-blocking, so a resource stays warm and consistent without any request waiting on another module to respond.

3. Functional Requirements

3.1 Authentication (FR-AUTH)

ID	REQUIREMENT	METHOD	STATUS
FR-AUTH-01	Email/password registration with bcrypt hashing	`POST /auth/signup`	Implemente
FR-AUTH-02	Email/password login with JWT issuance	`POST /auth/login`	Implemente
FR-AUTH-03	Google OAuth 2.0 (authorization code flow)	`GET /auth/google/login` → callback	Implemente
FR-AUTH-04	Shopify Customer Account OAuth (PKCE, no secret)	`GET /auth/shopify/login` → callback	Implemente
FR-AUTH-05	JWT-based session with configurable max age	`httpOnly` cookie, default 7 days	Implemente
FR-AUTH-06	Idle timeout with pre-logout warning	Configurable: 30min idle, 5min warning	Implemente
FR-AUTH-07	Password reset via email (Resend)	`POST /auth/forgot-password` → `POST /auth/reset-password`	Implemente
FR-AUTH-08	Profile update (name, language)	`POST /auth/profile`	Implemente

ID	REQUIREMENT	METHOD	STATUS
FR-AUTH-09	Account deletion (self-service)	`POST /auth/delete-account`	Implemente
FR-AUTH-10	Auth method toggles (admin-configurable)	KV config: `authMethods.email/google/shopify`	Implemente
FR-AUTH-11	Welcome email for Shopify-origin users	Auto-sent on customer create (webhook/cron); detects `!password_hash`	Implemente
FR-AUTH-12	Welcome vs reset password page variant	`?welcome=1` adjusts title/subtitle/button copy	Implemente
FR-AUTH-13	Shopify App OAuth with mandatory expiring tokens	`GET /auth/install` → `/auth/callback`; `expiring=1` (required since April 2026); scopes: `read_customers,write_customers,read_products,read_orders`; stores expiring access token + refresh token	Implemente
FR-AUTH-14	Webflow OAuth (replaces manual CMS token)	`GET /auth/webflow/connect` → `/auth/webflow/callback`	Implemente
FR-AUTH-15	OAuth state parameter CSRF protection	State generated, stored in KV, verified on callback, deleted after use	Implemente
FR-AUTH-16	Shop domain validation	Regex `^[a-zA-Z0-9][a-zA-Z0-9-]*\.myshopify\.com$` on install + callback	Implemente
FR-AUTH-17	Automatic Shopify token refresh	`refreshShopifyTokenIfNeeded()` called on `*/15 * * * *` cron; refreshes 5 min before expiry; rotates both access token and refresh token	Implemente
FR-AUTH-18	Manual Shopify token refresh	`POST /admin/shopify-refresh` — force-refreshes token, tests API, returns status; bearer auth required	Implemente

ID	REQUIREMENT	METHOD	STATUS
FR-AUTH-19	Shopify app scope management	Scopes declared in `shopify.app.crm-sync.toml` `[access_scopes]`; deployed via `npx shopify app deploy`; Shopify silently drops undeclared scopes	Implemente

3.2 Consent Management (FR-CONSENT)

ID	REQUIREMENT	STATUS
FR-CONSENT-01	TOS consent required on signup (configurable)	Implemented
FR-CONSENT-02	Cookie consent banner with accept/reject	Implemented
FR-CONSENT-03	Marketing opt-in (configurable)	Implemented
FR-CONSENT-04	CCPA sale-of-data opt-out	Implemented
FR-CONSENT-05	Newsletter subscription consent	Implemented
FR-CONSENT-06	Dynamic consent types from form bridge (any key)	Implemented
FR-CONSENT-07	All consent changes logged to `consent_records` audit trail	Implemented
FR-CONSENT-08	Known consent types written to `user_claims` table	Implemented
FR-CONSENT-09	Consent state visible in UCP Dashboard	Implemented
FR-CONSENT-10	Consent state synced to Shopify tags (`accepts_*` / `rejects_*`)	Implemented
FR-CONSENT-11	Consent state pushed to GA4 user properties	Implemented
FR-CONSENT-12	A2A Agent Access consent — controls AI agent read access via A2A protocol	Implemented
FR-CONSENT-13	AP2 Agent Payments consent — controls AI agent checkout via signed mandates	Implemented

ID	REQUIREMENT	STATUS
FR-CONSENT-14	AP2 consent gating — mandate creation returns 403 if `consent_ap2` not granted	Implemented

3.3 Tag System (FR-TAGS)

ID	REQUIREMENT	STATUS
FR-TAGS-01	Structured tags with categories: status, tier, segment, campaign, consent, marketing	Implemented
FR-TAGS-02	Tag auto-creation with category inference (`inferTagCategory`)	Implemented
FR-TAGS-03	Join table architecture (`crm_tags` + `user_tag_map`)	Implemented
FR-TAGS-04	Flat array on `storefront_users.tags` for backward compatibility	Implemented
FR-TAGS-05	Tags flow through full channel on every mutation	Implemented
FR-TAGS-06	Default tag seed: 17 tags across 6 categories	Implemented
FR-TAGS-07	Date-stamped campaign tags from form bridge (`{type}_YYYY-MM-DD`)	Implemented

3.4 CRM Sync (FR-SYNC)

ID	REQUIREMENT	STATUS
FR-SYNC-01	Shopify → Xano customer upsert (by GID or email match)	Implemented
FR-SYNC-02	Xano → Webflow CMS item upsert (by email match)	Implemented
FR-SYNC-03	Xano → Shopify metafields + tagsAdd	Implemented
FR-SYNC-04	Webflow CMS → Xano → Shopify (webhook-driven)	Implemented
FR-SYNC-05	Cron sync every 15 minutes with timestamp cursor	Implemented
FR-SYNC-06	Single-user sync on webhook (not full batch)	Implemented
FR-SYNC-07	Tag reference field linking Customers → CRM Tags collections	Implemented
FR-SYNC-08	Webflow collection auto-detect/create on init	Implemented
FR-SYNC-09	Force sync mode (bypass `lastRun` timestamp cursor)	`POST /sync/customers?force=1`
FR-SYNC-10	Standalone Webflow CMS sync endpoint	`POST /sync/webflow` (avoids 50-subrequest limit)
FR-SYNC-11	Shopify Admin webhook auto-registration on OAuth install	`webhookSubscriptionCreate` for `CUSTOMERS_CREATE` + `CUSTOMERS_UPDATE`

3.5 Analytics Integration (FR-GA4)

ID	REQUIREMENT	STATUS
FR-GA4-01	Server-side GA4 Measurement Protocol push	Implemented
FR-GA4-02	User properties: `crm_status`, `crm_tier`, `crm_segment`, `crm_tags`, `crm_campaign`, `consent_marketing`, `consent_tos`	Implemented
FR-GA4-03	Events: `crm_tags_updated`, `crm_sync`, `crm_form_submit`	Implemented
FR-GA4-07	UCP checkout events: `ucp_checkout_created`, `ucp_checkout_completed` with session_id, a2a_consent, ap2_consent	Implemented
FR-GA4-08	UCP mandate events: `ucp_mandate_created`, `ucp_mandate_used` with mandate_id, max_amount, scope	Implemented
FR-GA4-04	Client ID format: `crm-sync.{userId}` for server-originated events	Implemented
FR-GA4-05	`engagement_time_msec` > 0 for event visibility in reports	Implemented
FR-GA4-06	DataLayer push from form bridge for GTM tag pickup	Implemented

3.6 GDPR / Privacy (FR-GDPR)

ID	REQUIREMENT	STATUS
FR-GDPR-01	Customer redaction — anonymize PII across all Xano tables	Implemented
FR-GDPR-02	Data portability — compile all user data on request	Implemented
FR-GDPR-03	Shop redaction — acknowledge Shopify shop deletion	Implemented
FR-GDPR-04	Consent audit trail — append-only `consent_records` with timestamps	Implemented
FR-GDPR-05	Consent provenance — source, session ID, IP logged per change	Implemented
FR-GDPR-06	Right to deletion — `POST /auth/delete-account` (self-service)	Implemented
FR-GDPR-07	HMAC-SHA256 verification on all GDPR webhook handlers	`verifyShopifyHmac()` returns 401 for invalid signatures
FR-GDPR-08	Compliance webhook dispatcher at `/api/webhooks`	Routes by `X-Shopify-Topic` header; matches TOML `compliance_topics` URI

3.7 Form Bridge (FR-FORM)

ID	REQUIREMENT	STATUS
FR-FORM-01	Generic `data-crm-form` attribute on any HTML form	Implemented
FR-FORM-02	Auto-derives tag slug from attribute value	Implemented
FR-FORM-03	Creates `{type}_subscribed` + `{type}_YYYY-MM-DD` tags	Implemented
FR-FORM-04	Logs consent with form type as dynamic key	Implemented
FR-FORM-05	Fires `crm_form_submit` dataLayer event with form metadata	Implemented
FR-FORM-06	Programmatic API: `window._crmForms.submit()`	Implemented

3.8 Multi-Tenant Infrastructure (FR-TENANT)

ID	REQUIREMENT	STATUS
FR-TENANT-01	Per-shop KV config isolation (`tenant:{shop}:config`)	Implemented
FR-TENANT-02	Structured tenant registry (`tenants:index` as `TenantEntry[]` with shop, region, registered_at)	Implemented
FR-TENANT-03	Tenant identity resolution from headers/query/body	Implemented
FR-TENANT-04	`getTenantConfig(env, shop)` / `setTenantConfig(env, shop, config)`	Implemented
FR-TENANT-05	Legacy `crm_config` fallback for single-tenant (private plan) mode	Implemented
FR-TENANT-06	Cron iterates all registered tenants via `tenants:index`	Implemented
FR-TENANT-07	Per-tenant error isolation (one tenant failure does not block others)	Implemented
FR-TENANT-08	Tenant auto-registration on first config POST or OAuth install	Implemented
FR-TENANT-09	Platform-level config (`platform:config`) for shared credentials	Implemented
FR-TENANT-10	Region/market tenant groups with `inferRegionFromShop()` — geo-grouped localization (NA/EMEA/APAC/LATAM: US/CA/UK/DE/FR/AU/NZ/SG/JP/MX/BR). See §3.17.2 (FR-LOC)	Implemented

ID	REQUIREMENT	STATUS
FR-TENANT-11	`tenantKvKey(shop, key)` helper for all tenant-scoped KV operations	Implemented
FR-TENANT-12	Per-tenant admin keys (`admin_key` in tenant config) with platform key fallback	Implemented
FR-TENANT-13	UUID-keyed OAuth state (`oauth_state:{uuid}`) replacing global singleton keys	Implemented
FR-TENANT-14	`GET/POST /platform/config` for shared credential management	Implemented
FR-TENANT-15	`POST /admin/provision-region` for Xano schema replication per tenant	Implemented
FR-TENANT-16	`GET /admin/tenants?region=` with structured entries and region filter	Implemented
FR-TENANT-17	Backwards-compatible tenant registry migration (flat `string[]` → `TenantEntry[]`)	Implemented
FR-TENANT-18	Extension `?shop=` on all config POST, embed URL, and test endpoint fetch calls	Implemented

3.9 Adobe Experience Platform Integration (FR-ADOBE)

ID	REQUIREMENT	STATUS
FR-ADOBE-01	Adobe IMS OAuth server-to-server auth (`client_credentials` grant)	Implemented
FR-ADOBE-02	Per-tenant Adobe credentials in `CrmSiteConfig`	Implemented
FR-ADOBE-03	SHA-256 PII hashing (email, phone, name) via Web Crypto API	Implemented
FR-ADOBE-04	XDM ExperienceEvent schema mapping from Shopify customer data	Implemented
FR-ADOBE-05	Streaming ingestion to AEP dataset via `dcs.adobedc.net`	Implemented
FR-ADOBE-06	KV-cached IMS access tokens with TTL-based refresh	Implemented
FR-ADOBE-07	Sync status written to Xano `user_extras` (`adobe_sync_status` , `adobe_last_synced_at` , `adobe_ecid` , `adobe_email_hash`)	Implemented
FR-ADOBE-08	Adobe fields synced to Webflow CMS (ECID, email hash, sync status, last synced, identity graph ID)	Implemented
FR-ADOBE-09	Admin endpoint for Xano schema setup (`POST /admin/adobe-schema`)	Implemented
FR-ADOBE-10	Triggered on `handleWebhookCustomerUpdate` when `adobe_aep_enabled = true`	Implemented
FR-ADOBE-11	Granular XDM consent mapping from CRM tags (`marketing.email` , `marketing.push` , `personalize` , `adID`)	Implemented
FR-ADOBE-12	Subscription list parsing from CRM tags (`{type}_subscribed` → subscription objects with dates)	Implemented
FR-ADOBE-13	Direct form bridge → AEP `subscriptionEvent` push via `pushAdobeFormEvent` (no Shopify round-trip)	Implemented

ID	REQUIREMENT	STATUS
FR-ADOBE-14	Product upsell → AEP `commerce.productListAdds` event via `POST /webhooks/upsell`	Implemented
FR-ADOBE-15	Upsell endpoint auto-tags user (`upsell_{source}` , `upsell_{date}`), syncs to Shopify + GA4	Implemented

3.10 Pricing & Billing (FR-PRICING)

ID	REQUIREMENT	STATUS
FR-PRICING-01	Three-tier pricing model: Shared ($69/mo), Private ($325/mo), Enterprise (custom)	Implemented (UI)
FR-PRICING-02	Webflow extension Plan tab displays pricing cards	Implemented
FR-PRICING-03	"Check Subscription Status" queries tenant config for active features	Implemented
FR-PRICING-04	"Contact Sales" mailto link for Enterprise tier (`ysl@ysl150.com`)	Implemented
FR-PRICING-05	Integrations upgrade note (Salesforce, HubSpot, Klaviyo, Attentive, Braze)	Implemented
FR-PRICING-06	Stakeholder C (Private Worker) upgrade exception — see §3.10.1	Implemented
FR-PRICING-07	Stakeholder C self-service admin key rotation — see §3.10.2	Implemented

3.10.1 UPGRADE EXCEPTION: PRIVATE WORKER PURCHASERS (STAKEHOLDER C)

Stakeholders who deploy their own Cloudflare Worker instance ("Private Worker" tier) are **exempt from the Plan tab's locked-feature gating** because they control their own infrastructure. The following UI behaviors differ by stakeholder:

UI ELEMENT	STAKEHOLDER B (SHARED)	STAKEHOLDER C (OWN WORKER)
Plan tab — Current Plan	Shows "Shared" at $69/mo	Shows "Private" — detected by `plan` field in tenant config
Plan tab — pricing cards	All three tiers shown with upgrade path	Informational only — they already have full access
Custom Worker Setup	`[LOCKED]` — "Available on Private ($325/mo)"	Unlocked — Custom Worker URL and CDN Domain fields visible
OAuth redirect URIs	Fixed to shared worker domain	Editable — points to their own worker
"Upgrade plan to unlock →" (Config tab, Adobe section)	Links to Plan tab	Hidden — all integrations are self-managed
Setup Wizard — upgrade tip	"Upgrade to Private or Enterprise" link shown	Hidden or shows "You're on a self-hosted plan"
Subscription billing	Managed via Shopify App Billing (appears on merchant invoice)	**No Shopify billing** — billed separately (annual license or custom agreement)
Fees	$69/mo (Shared) or $325/mo (Private managed)	License fee per agreement — Cloudflare, Xano, and third-party costs are the customer's responsibility

Detection logic: The extension reads the `plan` field from `GET /config`. When `plan === "private"` or `plan === "enterprise"`, `updatePlanUI()` unlocks the Custom Worker Setup section and adjusts the badge. Stakeholder C sets this field in their own KV config during initial deployment.

Fee responsibility for Stakeholder C:

COST	WHO PAYS
CRM Sync license	Customer (per agreement with App Creator)
Cloudflare Workers (compute + KV)	Customer's Cloudflare account
Xano database	Customer's Xano account
Shopify app charges	Customer's Shopify account
Resend email	Customer's Resend account
Google OAuth / GA4	Customer's Google Cloud project
Adobe AEP (if enabled)	Customer's Adobe contract
Custom domain SSL	Customer's domain registrar

3.10.2 SELF-SERVICE ADMIN KEY MANAGEMENT (PERSONA C)

Private Worker operators (Stakeholder C) need admin key management **independent of any Shopify instance** to support multi-tenant Shopify deployments from a single worker. The system uses a two-tier key hierarchy:

Key hierarchy:

KEY	STORAGE	WHO SETS IT	WHO CAN ROTATE	SCOPE
Root key (`ADMIN_KEY`)	Cloudflare secret	App Creator (A) at deploy time via `wrangler secret put`	Only via Cloudflare CLI/API	Irrevocable bootstrap key
Rotatable key	KV (`admin_key:rotatable`)	Persona C via API	Persona C (self-service)	Runtime-rotatable, revocable by root

API endpoints:

METHOD	ENDPOINT	AUTH REQUIRED	DESCRIPTION
`POST`	`/admin/rotate-key`	Root OR current rotatable key	Generate or set a new rotatable key. Accepts optional `{ key: "..." }` (min 20 chars) or auto-generates 48-char hex. Previous rotatable key is immediately invalidated.
`GET`	`/admin/rotate-key`	Root OR current rotatable key	Returns metadata: `has_rotatable_key`, `created_at`, `rotated_at`, `rotations`, `key_preview` (first 6 + last 4 chars). Never returns the full key.
`DELETE`	`/admin/rotate-key`	Root key **only**	Revokes the rotatable key entirely. Only the root `ADMIN_KEY` can perform this action (prevents lockout).

Auth resolution order (applies to all admin endpoints):

1. Root `ADMIN_KEY` (Cloudflare secret) — always valid
2. Rotatable key (KV-stored) — valid when set, checked on every request
3. `SHOPIFY_APP_SECRET` — legacy fallback
4. Tenant tokens (`crm_t_*`) — shop-scoped, checked on config endpoints only

Multi-tenant Shopify use case: Persona C deploys one worker, connects multiple Shopify stores, and issues `crm_t_` tenant tokens per store. The rotatable admin key lets them manage all stores without needing Cloudflare CLI access. Key rotation doesn't affect existing tenant tokens.

Security constraints:

- Custom keys must be ≥ 20 characters
- Only one rotatable key exists at a time (rotation replaces the previous one)
- Root key cannot be rotated via API (requires `wrangler secret put`)
- Rotatable key revocation (`DELETE`) requires root key (prevents a compromised rotatable key from locking out the operator)
- Key metadata (rotation count, timestamps) is stored separately from the key value

3.11 UCP / A2A Protocol (FR-UCP)

ID	REQUIREMENT	STATUS
FR-UCP-01	UCP Discovery Manifest at `GET /.well-known/ucp` — dynamic capabilities, endpoints, auth methods	Implemented
FR-UCP-02	A2A Agent Card at `GET /.well-known/agent-card.json` — 5 skills, bearer auth, JSON output	Implemented
FR-UCP-03	Commerce Identity at `GET /commerce/identity` — unified profile with providers, consent, tags	Implemented
FR-UCP-04	Product Catalog at `GET /commerce/products?shop=` — Shopify Admin GraphQL, search, pagination, collection filter; `?shop=` required for multi-tenant product resolution	Implemented
FR-UCP-05	Order Management at `GET /commerce/orders` + `GET /commerce/orders/:id` — JWT auth, KV cache, ownership verification	Implemented
FR-UCP-06	Checkout Sessions — Storefront Cart API (primary) with Draft Order fallback; `POST /commerce/checkout`, `GET /commerce/checkout/:id`, `POST /commerce/checkout/:id/complete`	Implemented
FR-UCP-07	AP2 Payment Mandates — HMAC-SHA256 signed mandates with expiry; `POST /commerce/mandates`, `GET /commerce/mandates/:id`	Implemented
FR-UCP-08	Consent History at `GET /ucp/consent-history` — paginated audit trail	Implemented
FR-UCP-09	UCP Tag Sync at `POST /ucp/tags` — tags flow through full channel	Implemented
FR-UCP-10	CORS on all `/commerce/*` routes	Implemented

ID	REQUIREMENT	STATUS
FR-UCP-11	UCP manifest `checkout_api` field reflects active API (`storefront_cart` or `admin_draft_order`)	Implemented
FR-UCP-12	UCP manifest dynamically includes `a2a_agent_access` / `ap2_agent_payments` capabilities when consent enabled	Implemented

3.12 Embed System (FR-EMBED)

ID	REQUIREMENT	STATUS
FR-EMBED-01	Footer consent banner (`/embed/footer`) — GDPR-compliant banner with cookie preferences	Implemented
FR-EMBED-02	Compliance center (`/embed/compliance`) — data export, deletion, consent history	Implemented
FR-EMBED-03	Customer account (`/embed/account`) — profile, consent toggles (incl. A2A/AP2), linked providers, Protocol Status	Implemented
FR-EMBED-04	Customer dashboard (`/embed/dashboard`) — consent status, retarget channels, segment, tags, consent history, Protocol Status	Implemented
FR-EMBED-05	~~Product cart + checkout (`/embed/cart`)~~	Removed (v1.7) — storefront and cart handled by Shopify Web Components + PIM grid (`cf-worker-webflow-sync`)
FR-EMBED-06	`embeds_enabled` feature flag — tenant-configurable array controlling which embeds are served (default: all four)	Implemented
FR-EMBED-07	Disabled embeds return 403 with error message	Implemented
FR-EMBED-08	Shopify admin app **Embeds** tab (within the consolidated `/app` dashboard — see §3.15) — copy-ready snippets for all 4 embeds with copy-to-clipboard	Implemented

3.13 Storefront Token Provisioning (FR-SF)

ID	REQUIREMENT	STATUS
FR-SF-01	Auto-provision Storefront Access Token on Shopify app auth via `storefrontAccessTokenCreate`	Implemented
FR-SF-02	Reuse existing "CRM Sync Storefront" token if one exists (no duplicates)	Implemented
FR-SF-03	Storefront token sent to worker via `POST /config` on every app load	Implemented
FR-SF-04	Storefront token visible in Settings page (display + reset + manual override)	Implemented
FR-SF-05	Checkout auto-selects Storefront Cart API when token available, falls back to Draft Orders	Implemented

3.14 Token Lifecycle Management (FR-TOKEN)

3.14.1 SHOPIFY EXPIRING TOKEN REQUIREMENT

As of April 2026, Shopify mandates that all OAuth apps use expiring offline access tokens with rotation. Non-expiring tokens return `403`. All CRM Sync Admin API calls — customer sync, product queries, order lookups, webhook registration, and storefront token provisioning — require valid expiring OAuth tokens.

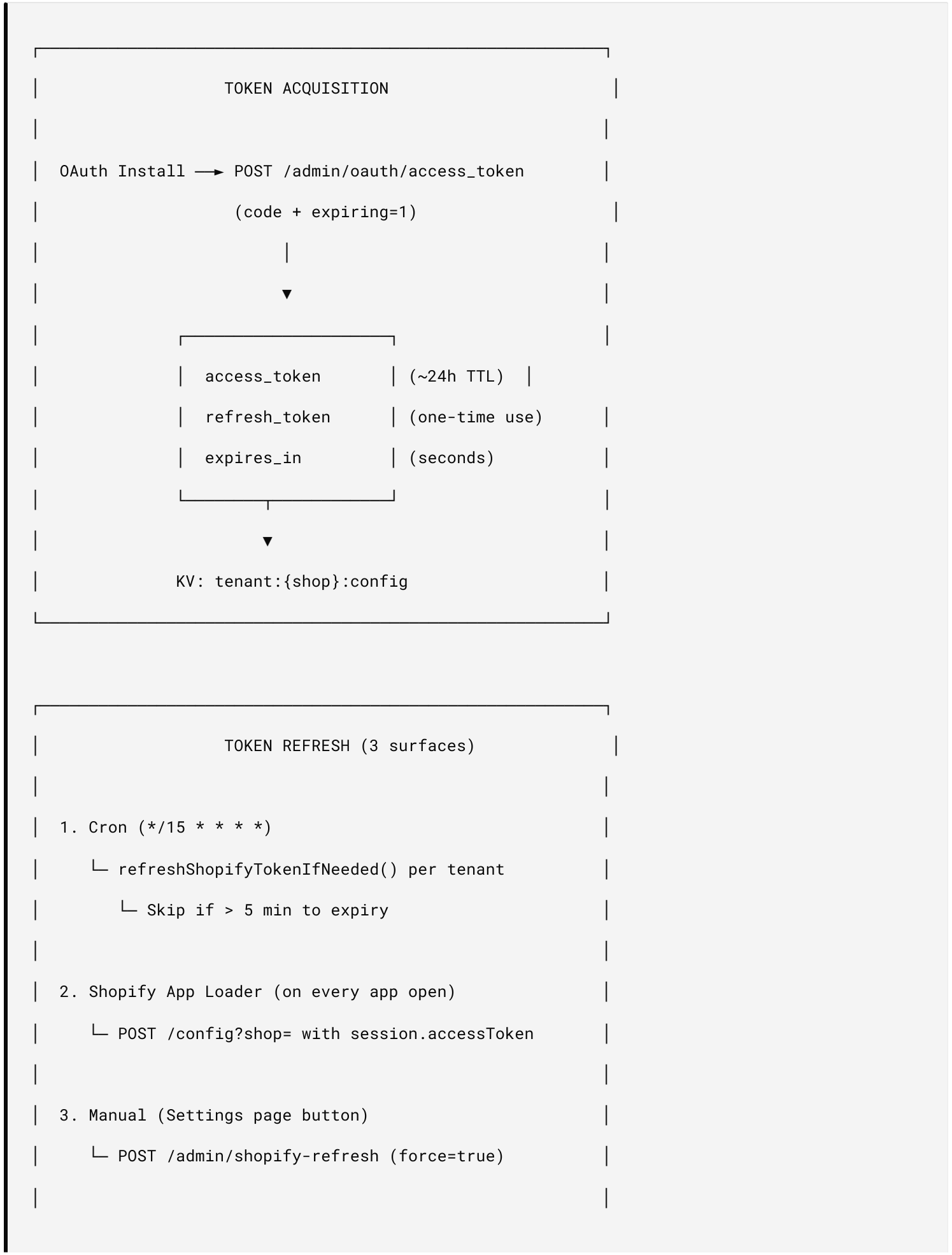
ASPECT	BEFORE (DEPRECATED)	AFTER (MANDATORY)
Token lifetime	Never expires	~24 hours (Shopify-controlled TTL)
Refresh token	Not issued	Issued alongside access token; rotates on each use
Token type	Non-expiring (no refresh)	Expiring (~~24h) with refresh token (~~90d)
Compliance flag	None	`expiring: "1"` in token exchange; `expiringOfflineAccessTokens: true` in app config

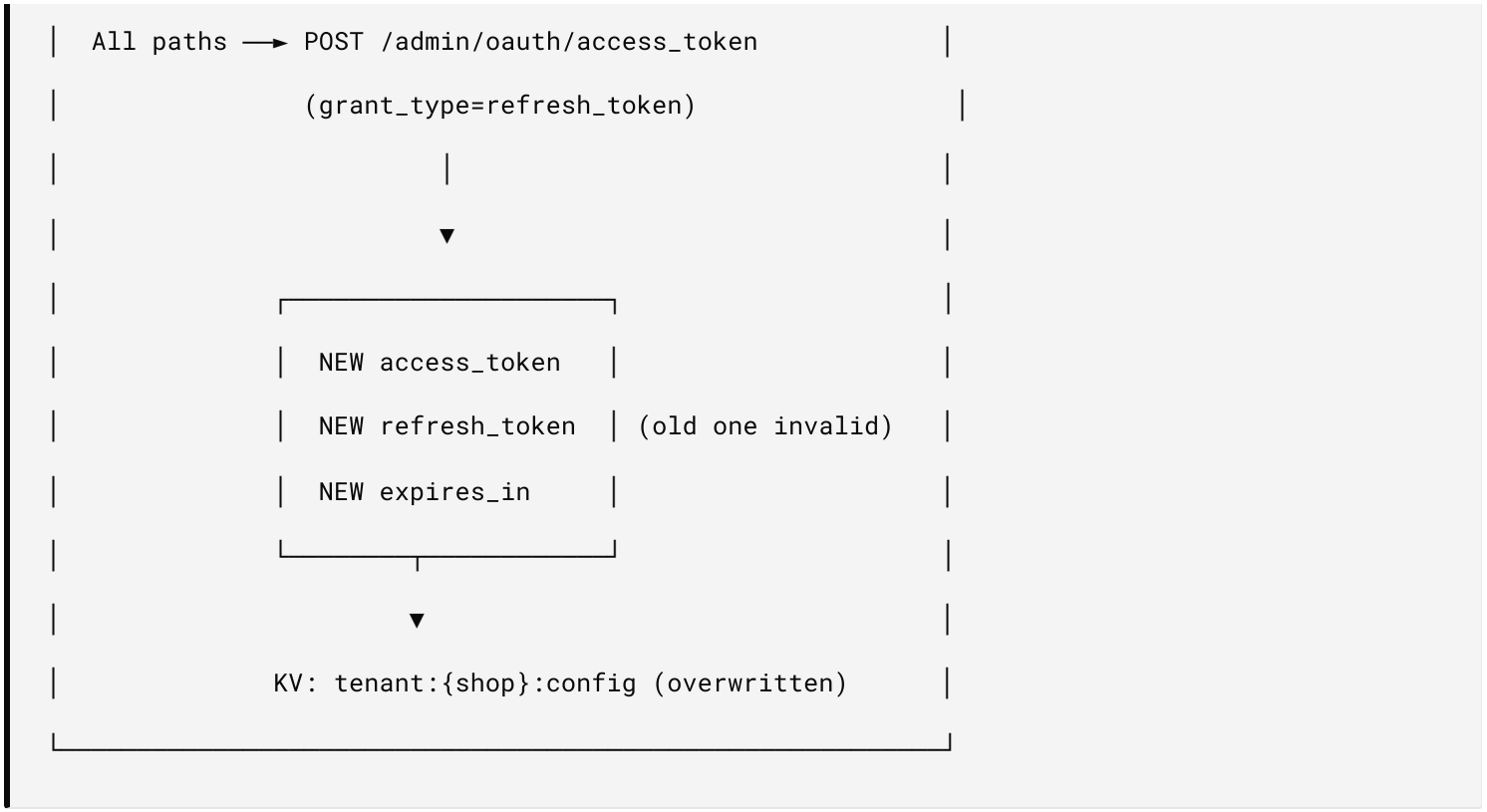
3.14.2 TOKEN REFRESH ARCHITECTURE

ID	REQUIREMENT	IMPLEMENTATION	STATUS
FR-TOKEN-01	OAuth install exchanges code with `expiring=1` flag	`POST /admin/oauth/access_token` with `expiring: "1"`	Implemented
FR-TOKEN-02	Store access token, refresh token, and expiry timestamp in tenant KV	`shopify_admin_token`, `shopify_refresh_token`, `shopify_token_expires_at`	Implemented
FR-TOKEN-03	Auto-refresh before expiry via cron (5-min buffer)	`refreshShopifyTokenIfNeeded()` called by `*/15 * * * *` scheduled handler	Implemented
FR-TOKEN-04	Rotate refresh token on each use (Shopify requirement)	New `refresh_token` from response saved back to KV	Implemented
FR-TOKEN-05	Shopify app session sync on every app open	`app.tsx` loader sends `session.accessToken` to CRM worker via `POST /config?shop=`	Implemented
FR-TOKEN-06	Manual force-refresh endpoint	`POST /admin/shopify-refresh` — bypasses expiry check, tests API, returns status	Implemented
FR-TOKEN-07	Pre-call refresh on Admin API queries	`shopifyAdminGql()` calls `refreshShopifyTokenIfNeeded()` before every GraphQL request	Implemented
FR-TOKEN-08	Multi-tenant token isolation	Each tenant's tokens stored independently under `tenant:{shop}:config`; cron iterates all tenants	Implemented
FR-TOKEN-09	Diagnostic endpoint for token health	`GET /admin/shopify-test` — tests API call, returns token prefix, length, domain, response	Implemented
FR-TOKEN-10	Settings page token status display	Shows token type (`OAuth (expiring)` / `Admin API`), refresh token presence, expiry countdown, API health	Implemented

ID	REQUIREMENT	IMPLEMENTATION	STATUS
FR-TOKEN-11	Client Credentials grant for own-store apps (no refresh token)	When a tenant has no `shopify_refresh_token`, `refreshShopifyTokenIfNeeded()` mints a 24h token via `grant_type=client_credentials`	Implemented
FR-TOKEN-12	Self-heal fallback when a refresh token is dead	A failed `grant_type=refresh_token` (HTTP 400 "requires an active refresh_token") clears the stored refresh token and falls through to the Client Credentials grant — recovery without re-install	Implemented
FR-TOKEN-13	Recover on 401 mid-request, not just before expiry	Public commerce endpoints force a token refresh and retry once on a `401/403`, so a token that fails before its recorded expiry still self-heals	Implemented

3.14.3 TOKEN FLOW DIAGRAM


TOKEN ACQUISITION
OAuth Install
POST /admin/oauth/access_token
(code + expiring=1)
access_token
(~24h TTL)
refresh_token
(one-time use)
expires_in
(seconds)
KV: tenant:{shop}:config
TOKEN REFRESH (3 surfaces)
1. Cron (*/15 * * * *)
refreshShopifyTokenIfNeeded() per tenant
Skip if > 5 min to expiry
2. Shopify App Loader (on every app open)
POST /config?shop= with session.accessToken
3. Manual (Settings page button)
POST /admin/shopify-refresh (force=true)



3.14.4 REQUIRED OAUTH SCOPES

SCOPE	PURPOSE	DECLARED IN
`read_customers`	Customer sync, identity lookup	`shopify.app.crm-sync.toml`
`write_customers`	Customer creation, tag/metafield writes	`shopify.app.crm-sync.toml`
`customer_read_customers`	Customer Account API reads	`shopify.app.crm-sync.toml`
`customer_write_customers`	Customer Account API writes	`shopify.app.crm-sync.toml`
`read_products`	Shop embed product grid	`shopify.app.crm-sync.toml`
`read_orders`	Order history in dashboard	`shopify.app.crm-sync.toml`

Scopes must be declared in `shopify.app.crm-sync.toml` under `[access_scopes]`. Shopify silently drops undeclared scopes. Deploy scope changes via `npx shopify app deploy --config=shopify.app.crm-sync.toml`.

3.14.5 EMBED LOADING PATTERN

All embed endpoints (`/embed/footer`, `/embed/account`, `/embed/dashboard`, `/embed/compliance`) use the fetch+innerHTML pattern with script re-execution. The Shopify app generates embed snippets with short variable names to prevent code editor line-wrapping:

```
<div id="crm-{embed}-embed"></div>
<script>
(function(){
  var W='{workerUrl}';
  fetch(W+'/embed/{embed}?shop={shop}')
    .then(function(r){return r.text()})
    .then(function(h){
      var c=document.getElementById('crm-{embed}-embed');
      if(!c)return;
      c.innerHTML=h;
      c.querySelectorAll('script').forEach(function(old){
        var ns=document.createElement('script');
        if(old.src){ns.src=old.src}
        else{ns.textContent=old.textContent}
        if(old.type){ns.type=old.type}
        old.parentNode.replaceChild(ns,old);
      });
    })
    .catch(function(e){console.error('CRM embed failed',e)});
})();
</script>
```

Account and dashboard endpoints support dual-format responses: `text/html` (default, for fetch+innerHTML) and `text/javascript` (when `Sec-Fetch-Dest: script`, for

`<script src>` loading). Standalone preview mode includes a Sign In fallback that redirects to Google OAuth when the footer embed is absent.

3.14.6 GRANT TYPES & SELF-HEAL RECOVERY

CRM Sync supports **two** Shopify token grant types, selected automatically per tenant. Both call `POST https://{shop}/admin/oauth/access_token`; the worker never stores or transmits long-lived static credentials in plaintext outside the tenant KV record.

GRANT	USED WHEN	TOKEN LIFETIME	REFRESH TOKEN	RECOVERY
Refresh Token (`grant_type=refresh_token`)	Merchant-installed apps (OAuth code flow)	~24h	Yes — single-use, rotates on every refresh	Re-install OAuth if the chain breaks
Client Credentials (`grant_type=client_credentials`)	Own-store / single-merchant apps with no refresh token	~24h	No — re-minted on demand	Automatic; no manual step

Selection logic — `refreshShopifyTokenIfNeeded()`:

1. If the tenant has a `shopify_refresh_token` **and** a recorded expiry, use the **Refresh Token** grant (skip if more than the 5-minute buffer remains, unless forced).
2. Otherwise, use the **Client Credentials** grant to mint a fresh 24h token.

Dead-refresh self-heal (FR-TOKEN-12). Shopify rotates refresh tokens single-use — each refresh returns a new one and invalidates the old. If a rotation's replacement is ever lost, the stored refresh token becomes permanently invalid and the grant returns `400 {"error":"invalid_request","error_description":"This request requires an active refresh_token"}` — even though the locally recorded expiry still looks valid. When this happens, the worker:

1. Logs the failure and **deletes** the dead `shopify_refresh_token` / `shopify_refresh_token_expires_at` from the tenant KV record.
2. **Falls through** to the Client Credentials grant, minting a working 24h token.
3. Persists the new token + expiry; subsequent refreshes use Client Credentials going forward.

This converts what was previously an outage requiring an OAuth re-install (symptom: all Shopify-backed tools — product search, cart, checkout, orders — silently return empty results) into a transparent recovery. Combined with the on-`401` retry (FR-TOKEN-13), a token that fails *before* its recorded expiry also recovers within a single request.

Failure reference

SYMPTOM	CAUSE	RESOLUTION
All product searches return zero results; cart/checkout/orders fail; UI otherwise healthy	Admin token invalid (expired or revoked)	Self-heals on next call (FR-TOKEN-13); else force-refresh
Refresh grant returns `400 … active refresh_token`	Refresh-token rotation chain broken	Auto-recovers via Client Credentials fallback (FR-TOKEN-12)
`403: Non-expiring access tokens`	Legacy non-expiring token	Re-install OAuth (expiring) or switch to Client Credentials
Client Credentials grant fails	Missing/invalid app client secret in tenant KV	Restore `shopify_app_secret` in tenant config

3.15 Shopify Admin Embedded App (FR-APP)

The merchant-facing control panel renders embedded in the Shopify admin (App Bridge), served by the React Router worker `hx-crm-sync` (`application_url` in `shopify.app.crm-sync.toml`). It is a single route (`/app`, `app/routes/app._index.tsx`) presenting three tabs.

ID	REQUIREMENT	STATUS
FR-APP-01	Single `/app` route renders the CRM Sync dashboard inside a native Polaris `s-page` (heading "CRM Sync")	Implemented
FR-APP-02	Three tabs — **Config**, **Embeds**, **Settings** — rendered as a native `s-button` strip (active tab `variant="primary"`, others `variant="tertiary"`)	Implemented
FR-APP-03	Active tab is driven by the `?tab=` query param (not client-only React state), so switching works via real navigation even before/without client-side hydration	Implemented
FR-APP-04	Tab links preserve all existing query params (`shop`, `host`, `embedded`, `id_token`) so App Bridge stays authenticated across the switch	Implemented
FR-APP-05	**Config** tab — Shopify token status cards (API connection, token, type, API test), Re-install OAuth + Open Worker Settings actions, and read-only integration config (secrets masked)	Implemented
FR-APP-06	**Embeds** tab — copy-ready fetch+innerHTML snippets for all 4 embeds with copy-to-clipboard (see FR-EMBED-08)	Implemented
FR-APP-07	**Settings** tab — update-config form (Xano, Google, Webflow, Resend, GA4, Shopify secret/storefront) and Danger Zone reset; form carries a hidden `intent=save` field so the native POST works pre-hydration	Implemented
FR-APP-08	Navigation is in-page tabs only — no `s-app-nav`; standalone `/app/embeds` and `/app/settings` routes are removed (consolidated into `/app`)	Implemented
FR-APP-09	Unauthenticated `GET /app` returns the Shopify embedded auth bounce (410/302), never a 404/500	Implemented

3.16 Agentic Commerce — Data Entitlement, Consent Engine & Enterprise Add-ons (FR-ENT)

Business goal. Agentic Commerce Checkout is the core product; per-Enterprise-Org data-layer / ERP integrations are billable add-on customization on top of it. The entitlement is both the **enforcement** mechanism and the **packaging/billing** mechanism — an Org unlocks add-ons by what its entitlement grants (`plan_tier` + `features[]`).

3.16.1 ENTITLEMENT CORE (SOURCE OF TRUTH)

ID	REQUIREMENT	STATUS
FR-ENT-01	Xano is the authoritative store of entitlements (tables 190–194): one row per `(subject_type, subject_ref)` where subject is `tenant` | `user` | `agent`, carrying `plan_tier`, `features[]`, `caps`, `consent`, lifecycle `state`, and a monotonic `state_version`	Implemented
FR-ENT-02	`GET /entitlement?subject_ref=` and `POST /entitlement` (admin-gated); state machine `pending → active → suspended → revoked` enforced (409 on illegal transition); `state_version` bumped on every write	Implemented
FR-ENT-03	Every entitlement write appends an ordered event to the change bus (`entitlement_changes`, int `id` = poll cursor) with the new snapshot + resolved target channels	Implemented
FR-ENT-04	A2A authorize is gated on `caps.a2a` (active entitlement) and the requested mandate is bounded to `caps.mandate_max_amount` before writing `linked_agents` (188)	Implemented
FR-ENT-05	AP2 checkout is gated on `caps.ap2` + a valid offline token + `amount ≤ mandate_max_amount`; order recorded in `agent_orders` (189) with `mandate_id`	Implemented
FR-ENT-06	Revoke cascade: revoking an entitlement emits a change → channels drop their projection → subsequent checkout is denied **offline**	Implemented

3.16.2 KEYS & OFFLINE TOKENS

ID	REQUIREMENT	STATUS
FR-ENT-10	EdDSA (Ed25519) signing keys; channels verify a signed entitlement token **offline** with the public key — no Xano round-trip on the hot path; tamper / expiry / alg-downgrade rejected	Implemented
FR-ENT-11	Non-destructive signing-key rotation `next → active → retired` with an overlap window: old-kid tokens verify until `not_after`; the retired private key is deleted immediately; zero downtime	Implemented
FR-ENT-12	Scoped `entitlement:read` config key (`XANO_ENTITLEMENT_KEY`) rides the high-volume read path; the master metadata key is never used for entitlement reads. Least-privilege; KV-editable + masked in the console	Implemented

3.16.3 OMNI-CHANNEL PROJECTION & VERSION-MONOTONIC CONSENT RESOLUTION

ID	REQUIREMENT	STATUS
FR-ENT-20	Channel adapters poll the change bus from a per-channel cursor (`channel_sync_state` , 194), apply, and advance the cursor — never backwards; a stale channel catches up purely from `?since={cursor}`	Implemented
FR-ENT-21	**Version-monotonic resolution:** each channel tracks the last-applied `state_version` per subject and applies a change only if strictly newer; older/equal versions are skipped. **Consent never regresses** regardless of arrival order or cadence; replay/re-delivery are idempotent no-ops	Implemented
FR-ENT-22	**Cadence classes:** `realtime` channels (Cloudflare, Shopify, GA4) project on every entitlement write; `batch` channels (Adobe, SAP, Nielsen) reconcile on the 15-min cron. Monotonic resolution makes the two cadences provably non-divergent	Implemented
FR-ENT-23	Channel adapters: **Cloudflare** (KV entitlement mirror + re-issued offline token), **Shopify** (customer/shop metafields + status/plan tags), **GA4/Google** (Consent Mode v2 + Signals audience properties)	Implemented
FR-ENT-24	`POST /channels/{channel}/sync` , `/state` , and `/replay {since}` (admin-gated) for operate/observe/recover	Implemented

3.16.4 FIRST-CLASS VERSIONED CONSENT

ID	REQUIREMENT	STATUS
FR-ENT-30	Consent is an explicit `consent` object on the entitlement (and in the offline token), versioned by `state_version`, so every channel projects the **same** consent	Implemented
FR-ENT-31	Google projection maps granular **Consent Mode v2** (`ad_user_data` / `ad_personalization` / `analytics_storage` / `ad_storage`): explicit consent wins per-signal, else state-derived (`revoked` / `suspended` → denied)	Implemented

3.16.5 ENTERPRISE ADD-ONS (PER-ORG, FEATURE-GATED)

ID	REQUIREMENT	STATUS
FR-ENT-40	Adobe AEP, SAP S/4HANA, NielsenIQ/Circana are `batch` channels that apply a change only if the entitlement's `features[]` includes the add-on key (`adobe_aep` / `sap` / `nielsen`); else skipped	Implemented
FR-ENT-41	Each add-on POSTs the canonical versioned entitlement+consent record to the Org's own connector endpoint (`addon_connectors[channel] = { url, key, enabled }`) — onboarding a new Org or stack is configuration, not code	Implemented

3.16.6 OPERATIONS CONSOLE (NON-DEVELOPER ACCESSIBLE)

ID	REQUIREMENT	STATUS
FR-ENT-50	`/settings` console (admin-gated, behind Cloudflare Access) surfaces: all credentials (masked, Set/Reset/reveal); entitlement core (active signing key + Generate/Rotate; per-channel cadence/cursor/status/last-synced + Sync); enterprise add-on connectors per Org; Header/Footer embed codes	Implemented
FR-ENT-51	All console config lives in KV and takes effect immediately on save — no deploy, no code change; defaults fall back to environment values; resets are non-destructive	Implemented
FR-ENT-52	`scripts/verify-entitlement.sh` exercises the full stack with the admin key (entitlement CRUD + state machine, sign/verify/tamper, rotation, channel sync/replay, revoke cascade) and reports PASS/FAIL; `--read-only` mode performs no mutations (safe on shared/prod)	Implemented

3.17 Multi-Tenant Scaling Conventions & Stakeholder Forward-Deploy Harness (FR-ENV / FR-LOC / FR-FDH)

Three orthogonal, configurable axes scale the multi-tenant platform; each is named and editable from the config portal (no code change).

3.17.1 DEPLOY CHANNELS (DEV / STAGE / PROD) + STAKEHOLDER MAPPING (FR-ENV)

ID	REQUIREMENT	STATUS
FR-ENV-01	Three deploy channels resolved per **request hostname** (one worker, many hostnames): `localhost` / `*dev*` → **Dev**, `*.workers.dev` → **Stage**, the production custom domain → **Prod/UAT**	Implemented
FR-ENV-02	Per-host **environment/deploy-team labels** (`environment_labels` : host → `{ env, team, note }`) editable in `/settings` ; resolved by `getEnvLabel` ; the Webflow Publish Refactor can own these values (publish target = environment)	Implemented
FR-ENV-03	Stakeholder → channel mapping (defaults): **Dev = Engineering**, **Stage = Agency Consulting Team**, **Prod/UAT = QA · Product Management / DPO · PMO · Deploy On-Prem Team**	Implemented
FR-ENV-04	`GET /env-label` (admin or tenant token) returns the current host's env/team; the Webflow extension renders an env·team chip + a Config Portal deep-link	Implemented

3.17.2 LOCALIZATION / MARKET WITH GEO SCALING (FR-LOC)

ID	REQUIREMENT	STATUS
FR-LOC-01	Localization is a geo-grouped market model: **NA** (US/CA), **EMEA** (UK/DE/FR), **APAC** (AU/NZ/SG/JP), **LATAM** (MX/BR) — each region → `{ geo, locale, languages, currency, country }`	Implemented
FR-LOC-02	Market inferred from shop **prefix or suffix** (`us-acme` , `acme-us` , `acme_au`) or country TLD (`.com.au` , `.co.uk` , `.co.nz` , `.com.mx` , …); overridable in `/settings` (geo-grouped selector)	Implemented
FR-LOC-03	Entitlement `caps.currency` defaults from the tenant's market (US→USD, UK→GBP, DE/FR→EUR, AU→AUD, JP→JPY, MX→MXN, …) when not set	Implemented
FR-LOC-04	Model scales by adding market entries under any geo without code-path changes	Implemented

3.17.3 DEPLOY → STATE MANAGEMENT BINDING (WEBFLOW UI + XANO DATA)

ID	REQUIREMENT	STATUS
FR-FDH-01	A deploy reaches state in two systems — **Webflow (UI)** and **Xano (Data)**. The portal env banner surfaces this environment's UI target (Webflow site) and Data target (Xano workspace)	Implemented

3.17.4 STAKEHOLDER FORWARD-DEPLOY HARNESS (FR-FDH)

The harness is the path that carries a change **forward through the three channels** while each stakeholder operates its own channel from the config portal — no code access required downstream.

```
Dev (Engineering)  ──►  Stage (Agency Consulting)  ──►  Prod / UAT (QA · Product Mgmt / DPO · P
   build + verify          configure + UAT prep              sign-off + production operation
        │                        │                                  │
        └─ each channel: own hostname + env/team label, own Webflow (UI) + Xano (Data) state ta
           operated via /settings (KV config, no deploy) and validated with verify-entitlement.
```

ID	REQUIREMENT	STATUS
FR-FDH-02	Each channel is self-identifying (env/team banner on `/settings` + `/setup`) and self-serviceable (portal config in KV, effective on save)	Implemented
FR-FDH-03	Promotion is forward-only across Dev → Stage → Prod; each channel binds its own Webflow (UI) + Xano (Data) state targets	Implemented
FR-FDH-04	Validation harness per channel: `verify-entitlement.sh` (full or `--read-only`) + the TDD static suite; `/setup` shows Agentic Commerce readiness with a Config Portal deep-link	Implemented
FR-FDH-05	Access control per stakeholder via Cloudflare Access (zero-trust) layered over the admin-key gate on the portal	Implemented

3.18 Modular Build Strategy & Dynamic Data Loops (FR-BUILD)

ID	REQUIREMENT	METHOD	STATUS
FR-BUILD-01	Each surface is an independently deployable module (separate worker, config, storage); no synchronous cross-module calls on the request path	Own `wrangler.toml` + KV per worker (`cf-worker-crm-sync` / `cf-worker-webflow-sync`)	Implemented
FR-BUILD-02	A storefront runs standalone (PIM-only or CRM-only); CRM embeds gated by `embeds_enabled` , disabled → empty `200` + `X-CRM-Embed-Disabled` (never an error body the loaders would inject as text)	`/embed/*` gate	Implemented
FR-BUILD-03	Cross-module data sharing is via dynamic loops, not runtime calls: outbound, failure-tolerant `content.changed` projection	`POST /events/content-drain`	Implemented
FR-BUILD-04	Read resources stay fresh via stale-while-revalidate + in-isolate single-flight (serve stale instantly, refresh in background; concurrent misses coalesce to one origin assembly)	`/product` , shop catalog, `/embed/footer`	Implemented
FR-BUILD-05	Headless build resources refresh via the export→ingest→publish loop (token-gated feed + content fingerprint cron + `repository_dispatch`)	`/export` , fingerprint cron	Implemented
FR-BUILD-06	Header/footer embeds are non-destructive: worker is source of truth, additive published contracts, revalidate-friendly cache, superseded UI hidden (not page-deleted); enforced by a static test suite	`header-footer-automation` skill + `header-footer-nondestructive` harness suite	Implemented

3.19 Chatbot FAQ Answer Cascade (FR-FAQ)

The conversational FAQ tool (`/commerce/faq`) answers from a **tiered cascade** — the first confident, locale-correct answer wins. The structured/AEO sources are tried first; the Botpress Knowledge Base (which merges the multilingual product PDFs with the FAQs) is a **locale-gated last-resort fallback**, never the primary source.

ID	REQUIREMENT	METHOD	STATUS
FR-FAQ-01	Answers resolve through an ordered cascade; first confident hit wins	Shopify KB → guide-list → weighted Xano FAQ → blog-intent → vectorized KB → Botpress KB → blog last-resort	Implemented
FR-FAQ-02	The vectorized KB (articles + PDF manuals) is **always filtered by the conversation's content locale** so foreign-language chunks never bleed into an answer	persona/market-resolved `contentLocale` on the vector query	Implemented
FR-FAQ-03	"List the guides/manuals" intent returns the full PDF catalog, not a single nearest match	guide-list intent → `kb_manuals` catalog (`answer_source: manual_list`)	Implemented
FR-FAQ-04	The Botpress KB is a **last-resort fallback**, reached only when every structured/AEO tier misses — it does not preempt locale-filtered answers	cascade step 2.5 (`!answer && faqs.length === 0`)	Implemented
FR-FAQ-05	Botpress passages are **locale-gated**: scripts the locale never uses are rejected (CJK/Hangul/Cyrillic/Arabic/Thai/kana; Vietnamese for non- `vi`), English requires predominantly-ASCII prose, plus a minimum-prose floor so bare title matches don't win	`passageLocaleOk()` + length floor	Implemented
FR-FAQ-06	The Botpress tier is feature-gated by the `BOTPRESS_PAT` secret; absent → the structured cascade runs alone (no hard dependency)	`cf-worker-crm-sync` secret	Implemented

3.20 Cost Architecture — Token-Maxing Resolution (FR-COST)

The business challenge — "token maxing." Agentic commerce runs on two metered meters: LLM **tokens** and rate-limited platform **APIs** (Shopify, GA4). Naïve agent stacks re-prompt the model to reason over raw data and round-trip that data through per-request / per-GB-**egress** cloud services — so cost scales with traffic and *maxes out* unpredictably. Token-maxing (runaway model +

egress spend) is the single biggest reason agentic deployments overrun budget.

The resolution. Push **Data Resolution** (entity joins, reconciliation, lookups, serving the resolved object) and **encryption / key logic** onto **Xano**, which bills a **fixed monthly fee with no egress and no per-request metering** (the Xano K8s/Docker/Redis data plane, §2.0). The LLM/agent loop stays thin and **cache-fed** — it consumes pre-resolved objects through MCP tools (the Anthropic *Tool Runner* loop is a **read-only consumer**, never re-deriving data by re-prompting). The unbounded-volume work lands on a **flat cost floor**; token spend stays bounded. This is what makes the flat FR-PRICING (§3.10) tiers defensible — the cost floor is fixed, not metered.

ID	REQUIREMENT	METHOD	STATUS
FR-COST-01	Data Resolution (joins, reconciliation, entity serving) executes on the fixed-fee Xano plane, never by LLM re-prompting	Xano Functions / Tasks / Triggers (§2.0)	Implemented
FR-COST-02	Repeat reads are served from fixed-cost caches so identical queries don't re-spend tokens / API quota	Cloudflare KV edge cache + Xano cache; stale-while-revalidate + single-flight (§2.5)	Implemented
FR-COST-03	Encryption + key-resolution ("key logic builders") run server-side **inside** the fixed-fee / no-egress boundary; keys never egress to metered services or the agent context	Xano-resident EdDSA signing keys + entitlement crypto (§3.16); Interactive Key Ceremony (§13)	Implemented
FR-COST-04	AI/API token spend is bounded per session — the agent consumes resolved objects via MCP tools; no unbounded re-prompting over raw data	MCP brain tools + Tool Runner consumer loop	Implemented
FR-COST-05	No-egress data movement: resolution, joins, and cross-app sync stay within the Xano plane (in / within / out is un-metered), so volume adds no egress cost	Xano fixed-fee model; cross-app sync via Xano outbox (PIM ⇄ CRM boundary)	Implemented / Planned
FR-COST-06	Cost is predictable per tier (flat Xano fee + bounded token budget) — the economic basis of the flat pricing tiers	cross-ref FR-PRICING (§3.10)	Spec

Why Xano specifically (vs metered cloud): fixed fee = predictable; no egress = data movement doesn't meter; managed Functions = the "encryption key logic builders" that keep crypto and key resolution *inside* the cost-flat, egress-free, governed boundary. The result pairs **cost predictability with the security boundary** — the heavy, unbounded data-resolution + crypto sits on one fixed-fee plane while the variable-cost model loop stays thin, so traffic growth doesn't multiply spend.

4. Data Model

4.1 Xano Tables

TABLE	ID	PURPOSE	F
`storefront_users`	180	User profiles, auth, tags	Y a
`user_claims`	181	Consent flags, auth provider details	Y t s
`user_extras`	182	Flexible per-user data (incl. Adobe tracking)	F E
`consent_records`	183	Append-only audit log	Y s
`crm_tags`	Dynamic	Tag definitions (name, slug, category)	N
`user_tag_map`	Dynamic	User ↔ Tag join table (source, timestamp)	N u
`adobe_sync_log`	187	Adobe AEP sync audit trail	Y E
`linked_agents`	188	Agentic record-keeping: linked agents + active mandate	N u
`agent_orders`	189	Agent order/checkout history with `mandate_id`	L r
`entitlements`	190	**Source of truth** — per `(subject_type, subject_ref)`: `plan_tier`, `features[]`, `caps`, `consent`, `state`, `state_version`	L p s
`entitlement_keys`	191	Config API key registry (hash only); non-destructive rolling	N h
`entitlement_signing_keys`	192	EdDSA keypairs for offline tokens (public key + `private_key_ref`; `next→active→retired`)	H r

TABLE	ID	PURPOSE	F
`entitlement_changes`	193	Ordered change event bus + **legal log** (append-only, int `id` = cursor); now records **actor attribution** — `actor`, `actor_type` (admin|agent|user|system), `mandate_id` — so the log answers who-did-what-to-whom	L s F
`channel_sync_state`	194	Per-channel reconcile cursor + status (`cloudflare` / `shopify` / `google` / `adobe` / `sap` / `nielsen`)	L

ENTITLEMENT `CONSENT` FIELD (TABLE 190)

Explicit, versioned consent object projected identically to every channel (Consent Mode v2 + agentic grants), e.g. `{ ad_user_data, ad_personalization, analytics_storage, ad_storage, marketing, a2a, ap2 }`. Bumps with `state_version` on every change.

ADOBE FIELDS ON `USER_EXTRAS` (TABLE 182)

FIELD	TYPE	DESCRIPTION
`adobe_ecid`	text	Adobe Experience Cloud ID assigned during AEP push
`adobe_email_hash`	text	SHA-256 hash of lowercase email (identity resolution)
`adobe_sync_status`	text	Last sync result: `success` or `error:{message}`
`adobe_last_synced_at`	text	ISO 8601 timestamp of last successful AEP push
`adobe_identity_graph_id`	text	Adobe Identity Graph reference ID

ADOBE SYNC LOG (`ADOBE_SYNC_LOG`, TABLE 187)

FIELD	TYPE	DESCRIPTION
`user_id`	integer	Foreign key to storefront_users
`email_hash`	text	SHA-256 hash of email
`ecid`	text	Adobe ECID
`dataset_id`	text	AEP dataset ID targeted
`sync_status`	text	`success` or `error`
`error_message`	text	Error detail (if any)
`created_at`	timestamp	Sync attempt time

4.2 Shopify Customer Metafields

NAMESPACE	KEY	TYPE	SOURCE
`custom`	`crm_status`	`single_line_text_field`	status-category tag
`custom`	`crm_tier`	`single_line_text_field`	tier-category tag
`custom`	`crm_segment`	`single_line_text_field`	segment-category tag
`custom`	`crm_tags`	`list.single_line_text_field`	All tag slugs
`custom`	`crm_consent_marketing`	`boolean`	`accepts_marketing` tag
`custom`	`crm_consent_tos`	`boolean`	`accepts_tou` tag presen
`entitlement`	`state`	`single_line_text_field`	Entitlement lifecycle state (projection)
`entitlement`	`plan_tier`	`single_line_text_field`	Entitlement plan tier
`entitlement`	`caps`	`json`	Entitlement caps (a2a/ap2/channels/mandat
`entitlement`	`consent`	`json`	Versioned consent object (s every channel sees)
`entitlement`	`state_version`	`number_integer`	Monotonic version (drives t regression guard)

> Customer-subject entitlements also project status/plan **tags** (`entitlement:active` , `plan:{tier}`); tenant-subject entitlements project the same fields as **shop** metafields via `metafieldsSet` .

4.3 Webflow CMS Collections

COLLECTION	FIELDS	AUTO-CREATED
CRM Tags	Name, Slug, Category, Source	Yes (init step)
Customers	Name, Email, First/Last Name, Provider, Status, Language, Tags, Orders, Amount Spent, Consent (TOS/Privacy/Cookie/Marketing), Subscriptions, Shopify ID, Country, Tag Refs, Adobe ECID, Adobe Email Hash, Adobe Sync Status, Adobe Last Synced, Adobe Identity Graph ID	Yes (init step)

4.4 KV Store Keys

MULTI-TENANT KEYS (CURRENT)

KEY PATTERN	CONTENTS	SENSITIVITY
`tenant:{shop}:config`	Per-tenant CrmSiteConfig (credentials, toggles, Adobe config, admin_key)	High — contains API keys
`tenant:{shop}:tag_table_ids`	Xano table IDs for crm_tags + user_tag_map (per-tenant)	Low
`tenant:{shop}:webflow_tags_collection_id`	Webflow CRM Tags collection ID (per-tenant)	Low
`tenant:{shop}:sync:customers:last_run`	ISO timestamp of last cron sync (per-tenant)	Low
`tenants:index`	`TenantEntry[]` — structured array with `{ shop, region, registered_at }`	Low
`platform:config`	Shared platform credentials (Shopify app secret, shared keys)	High
`pkce:{state}`	PKCE code_verifier (TTL, auto-expires)	Medium — OAuth state
`oauth_state:{uuid}`	OAuth state + shop + return_to (10min TTL, UUID-keyed)	Medium — CSRF nonce
`reset:{token}`	Password reset/welcome token: JSON `{ userId, welcome }` (1h/24h TTL)	Medium — auth token
`adobe_token:{shop}`	Cached Adobe IMS access token (TTL-based)	High — bearer token
`tenant:{shop}:checkout:{session_id}`	UCP checkout session state (24h TTL)	Medium — purchase data
`tenant:{shop}:mandate:{mandate_id}`	AP2 payment mandate with HMAC signature (per-expiry TTL)	Medium — purchase auth
`tenant:{shop}:orders:{customer_id}`	Cached order list (5min TTL)	Low — read cache
`signing:active_kid`	Active EdDSA signing key id	Medium

KEY PATTERN	CONTENTS	SENSITIVITY
`signing:sk:{kid}`	Private signing JWK (worker-held signer; never returned)	High — private key
`signing:pk:{kid}`	Public signing JWK (cached for offline verify; evicts at `not_after`)	Low — public key
`applied:{channel}:{subject_ref}`	Last-applied `state_version` per channel+subject (the version-monotonic consent guard)	Low
`ent:mirror:{subject_ref}`	Cloudflare projection of the entitlement snapshot	Low — permission/consent state
`ent:token:{subject_ref}`	Cached offline entitlement token (TTL = token exp)	Medium — signed token

All tenant-scoped KV keys are generated via the `tenantKvKey(shop, key)` helper, which returns `tenant:{shop}:{key}` when a shop is provided, or the bare `key` as fallback for legacy single-tenant mode.

LEGACY KEYS (SINGLE-TENANT / PRIVATE PLAN FALLBACK)

KEY	CONTENTS	SENSITIVITY
`crm_config`	Full site config (single-tenant mode)	High — contains API keys

5. Security Controls

5.0 Authentication & Gating Model

`ADMIN_KEY` is the **canonical gateway credential**. Setting your own `ADMIN_KEY` stands up a free, self-hosted instance; the shared and custom-DNS tiers are the monetized layers on top.

CREDENTIAL	USED BY	CHANNEL	NOTES
`ADMIN_KEY`	operator / admin tooling / both apps	`?key=` and `Bearer`	the gateway; canonical admin auth
Rotatable key (`admin_key:rotatable`)	self-service rotation (Persona C)	`Bearer` / `?key=`	set via `POST /admin/rotate-key` (records `admin_key:meta`)
Tenant token (`crm_t_…`)	Webflow extension / Stakeholder B	`Bearer`	shop-scoped, revocable (KV)
`SHOPIFY_APP_SECRET`	Shopify **webhook HMAC only**	—	**no longer an admin login** (dropped v1.13); the Shopify app now authenticates with `ADMIN_KEY`

- `keyEq` **trims** both sides of every comparison, so a stray trailing newline/space (e.g. a secret pasted into the Cloudflare dashboard Value box) never causes a false mismatch.
- **Cloudflare Access** (zero-trust) layers over the admin-key gate on the HTML admin pages (`/setup` , `/settings` , `/onboarding`) on the `*.workers.dev` host; the JSON API enforces the key directly.
- **Fail-open is local-dev only** — the gate opens only when *no* admin credential is configured at all (never in production, where `ADMIN_KEY` is set).

5.1 Authentication Security

CONTROL	IMPLEMENTATION
Password hashing	bcrypt (via Web Crypto API)
Session tokens	JWT with `HS256`, configurable expiry
Token delivery	`httpOnly`, `Secure`, `SameSite=None` cookie
OAuth state (user auth)	Random nonce stored in KV (`oauth_state:{state}`), verified on callback, deleted after use
OAuth state (app install)	Random UUID stored in KV (`oauth_state:{uuid}`), verified in callback, rejected with 403 on mismatch; includes shop domain to prevent cross-tenant collisions
PKCE	S256 code challenge (Shopify Customer Account)
Shop domain validation	`validateShopDomain()` regex on install + callback; rejects non-`.myshopify.com` domains
Idle timeout	Client-side timer with server-validated token expiry
Expiring offline tokens	Mandatory since April 2026; `expiring=1` in token exchange; expiring access token (Shopify-controlled TTL) + refresh token stored; auto-refreshed via cron with 5-min buffer; `POST /admin/shopify-refresh` for manual force-refresh

5.2 API Security

CONTROL	IMPLEMENTATION
Protected endpoints	JWT verification required for `/ucp/*`, `/auth/me`, `/auth/profile`, `/auth/delete-account`, `/tags/user`, `/segment/*`
Config endpoint auth	`POST /config` requires `Authorization: Bearer <ADMIN_KEY>` header; checks `ADMIN_KEY` or `SHOPIFY_APP_SECRET` env var
Admin endpoint auth	All `/admin/*` and `/sync/*` endpoints require `Authorization: Bearer <ADMIN_KEY>` header
Per-tenant admin keys	Tenants may set `admin_key` in their config; post-resolution auth gates check tenant key first, platform key as fallback
Tenant isolation	Config reads/writes scoped to `tenant:{shop}:config`; all KV operations use `tenantKvKey()` helper; no cross-tenant data access
CORS	Origin whitelist via `auth_redirect_origin` config
Secret masking	`GET /config` masks all credentials (first 4 + last 4 chars)
Embed HTML isolation	`getPublicCrmConfig()` strips API keys, tokens, and secrets before injecting config into client-side embed HTML (`/embed/footer`, `/embed/compliance`)
Webhook HMAC (Shopify)	`verifyShopifyHmac()` validates `X-Shopify-Hmac-SHA256` on all 5 webhook/GDPR handlers; returns 401 for invalid signatures
Compliance dispatcher	`/api/webhooks` route dispatches by `X-Shopify-Topic` header, matching TOML `compliance_topics` URI
Webhook HMAC (Webflow)	Webflow OAuth callback verifies `state` parameter against KV-stored nonce
OAuth callback redirect	OAuth callback auto-redirects to `/onboarding?shopify=connected` instead of rendering diagnostic HTML
PII logging prevention	No PII logged to browser console or server logs
Config isolation	KV > env var precedence prevents accidental secret exposure

5.3 Network Security

CONTROL	IMPLEMENTATION
Cloudflare Access (Zero Trust)	Email-based OTP access policy on worker admin URLs; whitelist: `ysl@ysl150.com`, `*@story-story.ai`
Access auth domain	`kcoop.cloudflareaccess.com`
CRM Sync Access App	Protects `crm.story-story.ai`
PIM Sync Access App	Protects `cf-worker-webflow-sync.yoonsunlee150.workers.dev`
Identity provider	One-Time Pin (OTP) — email verification, no external IdP dependency

5.4 Data Protection

CONTROL	IMPLEMENTATION
Encryption in transit	HTTPS enforced (Cloudflare edge)
Encryption at rest	Cloudflare KV encryption, Xano managed encryption
PII minimization	GA4 receives tag categories, not raw PII
Adobe PII hashing	SHA-256 hash of email/phone/name before AEP transmission; raw PII never leaves worker
Client-side PII	No `console.log` of user data (email, name, tokens) in embed scripts
Data retention	Consent records: append-only, no TTL (audit requirement)
Right to erasure	Full PII anonymization across all tables on redact
Data portability	`POST /gdpr/data-request` compiles complete user record
Token security	Refresh tokens stored in KV; access tokens auto-refreshed before expiry; non-expiring tokens return 403 since April 2026
Adobe token caching	IMS access tokens cached in KV with TTL; auto-refreshed on expiry

6. Consent Architecture

6.1 Consent Flow

```
User Action (toggle, form, signup)
  │
  ├─► consent_records (append-only audit entry)
  │     └─ user_id, type, granted/revoked, timestamp, source, session_id, ip
  │
  ├─► user_claims (known columns: tos, privacy, cookie, marketing, newsletter, a2a, ap2)
  │
  ├─► Shopify tags (accepts_{type} / rejects_{type})
  │
  └─► GA4 user properties (consent_marketing, consent_tos)
```

6.2 Consent Types

TYPE	STORAGE COLUMN	SHOPIFY TAG	GA4 PROPERTY	CONFIGURABLE
TOS	`consent_tos`	`accepts_tou`	`consent_tos`	Yes
Privacy	`consent_privacy`	`accepts_privacy`	—	Yes
Cookie	`consent_cookie`	`accepts_cookie`	—	Yes
Marketing	`consent_marketing`	`accepts_marketing`	`consent_marketing`	Yes
Newsletter	`consent_newsletter`	`newsletter_subscribed`	—	Yes
CCPA	`consent_ccpa`	—	—	Yes
A2A Agent Access	`consent_a2a`	`accepts_a2a`	—	Yes
AP2 Agent Payments	`consent_ap2`	`accepts_ap2`	—	Yes
Dynamic (form)	Audit trail only	`accepts_{type}`	—	Automatic

6.3 Audit Trail Schema (`consent_records`)

FIELD	TYPE	DESCRIPTION
`user_id`	integer	Foreign key to storefront_users
`consent_type`	string	e.g., `tos` , `marketing` , `waitlist`
`granted`	boolean	true = granted, false = revoked
`source`	string	`signup` , `ucp` , `form_bridge` , `api`
`session_id`	string	GA4 session ID (if available)
`ip_address`	string	Request IP (for legal provenance)
`created_at`	timestamp	Immutable creation time

6.4 Consent Control Plane vs. Consumers (Clean Room relationship)

The entitlement/consent engine (§3.16) is the **consent control plane** — the authoritative, versioned source of who consented to what, propagated to every channel with the version-monotonic guarantee that consent never regresses across cadences.

Other subsystems are **consumers** of that consent rather than independent owners of it:

- **Clean Room** (`CLEAN-ROOM-SPEC.md`) is a *downstream consumer*, not a competing reconciliation. It is a privacy-preserving data **match** (D1 + R2 + Durable Objects → non-reversible aggregates), a fundamentally different concern from consent **state propagation**. Its `clean_room` consent scope, consent verification at export (§5.3), and revocation flow (§5.4) should be driven by the versioned consent: gate a record's inclusion on `consent.clean_room == granted` at the current `state_version`, so the revoke cascade automatically excludes withdrawn records from future match jobs. `clean_room` is a candidate future `batch` channel.
- **Distinct vendor roles:** NielsenIQ/Circana appear as Clean Room *match partners* (compare hashed datasets → aggregates) **and** as add-on *projection channels* (push the versioned entitlement/consent record to their ingestion endpoint) — two separate integrations with the same vendor.

CONCERN	OWNER
Authoritative consent state + propagation (no regression, mixed cadence)	Consent control plane (§3.16)
Privacy-preserving cross-party match → aggregates (no PII egress)	Clean Room (consumer)

7. UAT Test Plan

7.1 Test Environment

ITEM	VALUE
Worker URL	`https://crm.story-story.ai`
Test Site	`https://omenphase1-1.webflow.io`
Shopify Store	`hx-stage.myshopify.com`
GA4 Property	`G-S7QGFWPZ8X`
Test Date	2026-05-____
Tester	_____

7.2 Pre-UAT Checklist

#	ITEM	STATUS	NOTES
PRE-01	Worker deployed and `/health` returns `{"status":"ok"}`	[ ]	
PRE-02	All config credentials populated (`GET /config` , no empty values)	[ ]	
PRE-03	`google_client_id` format valid (contains `-` after project number)	[ ]	
PRE-04	`shopify_admin_token` is a valid OAuth token (not an App automation token)	[ ]	
PRE-05	`webflow_collection_id` points to Customers collection (not Tags)	[ ]	
PRE-06	GA4 Measurement ID format `G-XXXXXXXXXX`	[ ]	
PRE-07	GTM container installed on Webflow site	[ ]	
PRE-08	Webflow webhook registered (`collection_item_changed`)	[ ]	

7.3 Authentication Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-AUTH-01	Email signup	1. Visit site 2. Click login 3. Switch to signup 4. Enter email/password 5. Accept TOS	Account created, JWT cookie set, user appears in Xano	[ ]	
UAT-AUTH-02	Email login	1. Click login 2. Enter credentials	JWT issued, nav shows user name/avatar	[ ]	
UAT-AUTH-03	Google OAuth	1. Click "Sign in with Google" 2. Complete Google consent	User created/linked, redirected to site	[ ]	
UAT-AUTH-04	Shopify PKCE login	1. Click "Sign in with Shopify" 2. Authorize on Shopify	User created/linked via PKCE flow	[ ]	
UAT-AUTH-05	Password reset	1. Click "Forgot password" 2. Enter email 3. Check inbox 4. Click link 5. Set new password	Email received, password changed, can login	[ ]	
UAT-AUTH-06	Idle timeout	1. Login 2. Wait for idle timeout 3. Observe warning	Warning appears at configured time, logout after expiry	[ ]	
UAT-AUTH-07	Profile update	1. Login 2. Go to Account page 3. Change name 4. Save	Name updated in Xano + nav display	[ ]	
UAT-AUTH-08	Account deletion	1. Login 2. Account page 3. Delete account 4. Confirm	Account deleted, logged out, cannot login	[ ]	
UAT-AUTH-09	Welcome email (Shopify-origin)	1. Create customer in Shopify 2. Trigger webhook/sync	Welcome email sent with "Set Your Password" link (24h TTL token)	[ ]	

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-AUTH-10	Welcome password set	1. Click welcome link 2. Set password	Page shows "Welcome" variant (not "Reset"), password saved, can login	[ ]	
UAT-AUTH-11	Shopify App OAuth install	1. `GET /auth/install?shop=store.myshopify.com`	Redirects to Shopify OAuth with state parameter	[ ]	
UAT-AUTH-12	Shopify App OAuth callback	1. Complete OAuth approval	Token exchanged, KV config updated, webhooks registered, redirect to onboarding	[ ]	
UAT-AUTH-13	Webflow OAuth connect	1. `GET /auth/webflow/connect`	Redirects to Webflow OAuth with state parameter	[ ]	
UAT-AUTH-14	Webflow OAuth callback	1. Complete Webflow approval	Token stored in KV config, site ID saved	[ ]	

7.4 Consent Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-CON-01	TOS consent on signup	1. Signup without checking TOS	Signup blocked with error	[ ]	
UAT-CON-02	Cookie banner	1. Visit site (new session)	Cookie banner appears	[ ]	
UAT-CON-03	Cookie accept	1. Click accept on banner	Banner dismissed, `consent_cookie: granted` in audit trail	[ ]	
UAT-CON-04	Cookie reject	1. Click reject on banner	Banner dismissed, `consent_cookie: revoked` in audit trail	[ ]	
UAT-CON-05	Marketing opt-in	1. Login 2. Dashboard 3. Toggle marketing on	Consent logged, Shopify tag `accepts_marketing` added	[ ]	
UAT-CON-06	Marketing opt-out	1. Toggle marketing off	Consent logged as revoked, Shopify tag removed	[ ]	
UAT-CON-07	Consent audit trail	1. Make consent changes 2. Check `/ucp/consent-history`	All changes listed with timestamps, source, session	[ ]	
UAT-CON-08	CCPA opt-out	1. Visit compliance page 2. Toggle "Do Not Sell"	CCPA consent logged	[ ]	

7.5 Tag Channel Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-TAG-01	Add tag from UCP	1. Login 2. Dashboard 3. Add "new_campaign" tag	Tag appears in: Xano join table, Shopify customer tags, Webflow CMS, GA4 user properties	[ ]	
UAT-TAG-02	Remove tag from UCP	1. Remove tag from Dashboard	Tag removed from all channels	[ ]	
UAT-TAG-03	Tag auto-creation	1. Add tag with unknown slug (e.g., "summer_sale")	Tag created in `crm_tags` with category "campaign"	[ ]	
UAT-TAG-04	Tag category inference	1. Add "vip" tag	Category assigned as "tier" (not "campaign")	[ ]	
UAT-TAG-05	Cron sync	1. Add tag in Shopify Admin 2. Wait 15 min (or manual trigger)	Tag appears in Xano + Webflow CMS	[ ]	
UAT-TAG-06	Shopify webhook → Xano + Webflow	1. Edit customer tags in Shopify	Webhook fires, tags synced to Xano join table + Webflow CMS item	[ ]	
UAT-TAG-07	Webflow CMS → Xano + Shopify	1. Edit customer tags in Webflow CMS	Webhook fires, tags synced to Xano + Shopify metafields	[ ]	

7.6 Form Bridge Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-FORM-01	Newsletter form	1. Add `data-crm-form="newsletter"` form to page 2. Submit with email	Tags created: `newsletter_subscribed` + `newsletter_2026-05-14`. Consent logged. DataLayer event fired.	[ ]	
UAT-FORM-02	Waitlist form	1. Add `data-crm-form="waitlist"` form 2. Submit	Tags: `waitlist_subscribed` + `waitlist_2026-05-14`	[ ]	
UAT-FORM-03	Custom form type	1. Add `data-crm-form="demo_request"` 2. Submit	Tags: `demo_request_subscribed` + date tag. Dynamic consent logged.	[ ]	
UAT-FORM-04	GTM event pickup	1. Submit any CRM form 2. Check GTM debug mode	`crm_form_submit` event with `form_type`, `email`, `session_id` parameters	[ ]	
UAT-FORM-05	Unauthenticated form	1. Submit form without being logged in	DataLayer event fires, no CRM tag creation (graceful skip)	[ ]	

7.7 GA4 Integration Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-GA4-01	User properties on tag change	1. Add tags 2. Check GA4 DebugView	User properties (`crm_status` , `crm_tier` , etc.) appear	[ ]	
UAT-GA4-02	crm_tags_updated event	1. Add/remove tag 2. Check GA4 DebugView	Event with `tags_added` / `tags_removed` params	[ ]	
UAT-GA4-03	crm_sync event (cron)	1. Trigger manual sync 2. Check GA4	`crm_sync` event with `source: shopify_cron`	[ ]	
UAT-GA4-04	crm_form_submit event (GTM)	1. Submit CRM form 2. Check GA4 Realtime	Event with form_type, email hash	[ ]	
UAT-GA4-05	Consent in user properties	1. Grant marketing consent 2. Check GA4	`consent_marketing: granted` in user properties	[ ]	

7.8 GDPR / Privacy Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOT
UAT-GDPR-01	Customer redaction	1. `POST /gdpr/customer-redact` with email	All PII anonymized: email → `redacted_{id}@redacted.local`, name cleared, tokens cleared, status → "redacted"	[ ]	
UAT-GDPR-02	Data request	1. `POST /gdpr/data-request` with email	Complete JSON export: user profile, claims, extras, consent records, tags	[ ]	
UAT-GDPR-03	Shop redaction	1. `POST /gdpr/shop-redact`	Acknowledged response	[ ]	
UAT-GDPR-04	Audit trail immutability	1. Redact user 2. Check consent_records	Consent records preserved (not deleted) after user redaction	[ ]	
UAT-GDPR-05	Self-service deletion	1. Login 2. Account → Delete 3. Confirm	Account removed, session cleared, login fails	[ ]	
UAT-GDPR-06	Consent history access	1. Login 2. `GET /ucp/consent-history`	Full audit trail returned for authenticated user	[ ]	
UAT-GDPR-07	Compliance dispatcher	1. `POST /api/webhooks` with `X-Shopify-Topic: customers/data_request` and valid HMAC	Data compiled and returned (same as `/gdpr/data-request`)	[ ]	
UAT-GDPR-08	Compliance unknown topic	1. `POST /api/webhooks` with `X-Shopify-Topic: unknown/topic`	`400 Unknown webhook topic`	[ ]	

7.9 Sync Integration Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-SYNC-01	Full cron cycle	1. `POST /sync/customers`	Customers synced: Shopify → Xano → Webflow → GA4. Response includes `synced` count.	[ ]	
UAT-SYNC-02	New customer from Shopify	1. Create customer in Shopify 2. Trigger sync	Customer appears in Xano + Webflow CMS	[ ]	
UAT-SYNC-03	Webflow → Shopify tag sync	1. Edit customer tags in Webflow CMS 2. Check Shopify customer	Tags + metafields updated on Shopify customer	[ ]	
UAT-SYNC-04	Shopify → Webflow tag sync	1. Add tag in Shopify Admin 2. Trigger sync	Tag appears in Webflow CMS customer item	[ ]	
UAT-SYNC-05	Collection auto-creation	1. Delete Customers collection 2. Run `POST /admin/init-tag-system?step=webflow`	Customers collection re-created with all fields, ID saved to config	[ ]	

7.10 Security Tests

ID	TEST CASE	STEPS	EXPECTED RESULT
UAT-SEC-01	Unauthenticated UCP access	1. `GET /ucp/consent-history` without JWT	`401 unauthorized`
UAT-SEC-02	Invalid JWT	1. Send request with tampered JWT	`401 unauthorized`
UAT-SEC-03	Config secrets masked	1. `GET /config`	All API keys/tokens show `xxxx••••xxxx` format
UAT-SEC-04	CORS enforcement	1. Request from unauthorized origin	CORS headers restrict respons
UAT-SEC-05	Password not stored in plain text	1. Check Xano user record	`password_hash` field contai bcrypt hash, no plaintext
UAT-SEC-06	Webhook HMAC enforcement	1. `POST /webhooks/customer-update` without HMAC header	`401 Invalid HMAC signat` (when app secret configured)
UAT-SEC-07	GDPR HMAC enforcement	1. `POST /gdpr/customer-redact` with invalid HMAC	`401 Invalid HMAC signat`
UAT-SEC-08	Compliance dispatcher HMAC	1. `POST /api/webhooks` with `X-Shopify-Topic: customers/redact` without valid HMAC	`401 Invalid HMAC signat`
UAT-SEC-09	OAuth state verification	1. `GET /auth/callback?code=xxx&shop=test.myshopify.com&state=wrong`	`403 Invalid state parameter`
UAT-SEC-10	Shop domain validation	1. `GET /auth/install?shop=evil.example.com`	`400 Invalid shop domain`

ID	TEST CASE	STEPS	EXPECTED RESULT
UAT-SEC-11	Config endpoint auth	1. `POST /config` without Authorization header	`401 Unauthorized`
UAT-SEC-12	Config endpoint auth (valid)	1. `POST /config` with `Authorization: Bearer <ADMIN_KEY>`	`200 { ok: true }`
UAT-SEC-13	Embed HTML secrets stripped	1. `GET /embed/footer` 2. Inspect HTML source	No API keys, tokens, or secret page source; only authMethod session, consent, domains
UAT-SEC-14	No PII in console	1. Login 2. Open browser DevTools Console 3. Navigate to dashboard	No email, name, or user data logged to console
UAT-SEC-15	OAuth callback redirect	1. Complete Shopify App OAuth	302 redirect to `/onboarding shopify=connected` (no diagnostic HTML with token details)
UAT-SEC-16	Admin endpoint auth gate	1. `POST /admin/adobe-schema` without `Authorization` header	`401 Unauthorized`
UAT-SEC-17	Sync endpoint auth gate	1. `POST /sync/customers` without `Authorization` header	`401 Unauthorized`
UAT-SEC-18	Admin endpoint valid auth	1. `POST /admin/webflow-ensure-fields?shop=x` with valid bearer	`200` with fields result
UAT-SEC-19	Cloudflare Access blocks unauthenticated browser	1. Open worker admin URL in incognito browser	Redirected to `kcoop.cloudflareaccess.` OTP login
UAT-SEC-20	Cloudflare Access allows whitelisted email	1. Authenticate with `ysl@ysl150.com` via OTP	Access granted to worker adm pages

ID	TEST CASE	STEPS	EXPECTED RESULT
UAT-SEC-21	Mandate read fail-closed (no auth)	1. `GET /commerce/mandates/{id}` without `Authorization` header	`401 unauthorized` — never leaks `404` mandate existenc (regression guard: `handleMandateGet` must pa the real `env` to `verifyBearerToken`, not a synthetic `{ADMIN_KEY}` env)
UAT-SEC-22	Mandate read fail-closed (bad bearer)	1. `GET /commerce/mandates/{id}` with `Authorization: Bearer not-a-real-key`	`401 unauthorized`
UAT-SEC-23	Permission gates reject invalid bearer	1. `GET /config`, `POST /config`, `POST /sync/customers`, `GET /admin/tenants` each with a bogus bearer	All `401` (token ≠ `ADMIN_KEY`/tenant/rotatable key)

> Automated coverage for UAT-SEC-21..23 and the mandate gates lives in `tests/smoke-test.sh` (`npm run test:smoke`) under **Mandates & Permissions (fail-closed)** — checks `MND-01..04` and `PERM-01..04`.

7.11 Multi-Tenant Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-MT-01	Tenant config isolation	1. `POST /config?shop=shop-a.myshopify.com` with config A 2. `POST /config?shop=shop-b.myshopify.com` with config B 3. `GET /config?shop=shop-a`	Config A returned (not B); KV key is `tenant:shop-a.myshopify.com:config`	[ ]	
UAT-MT-02	Tenant auto-registration	1. `POST /config?shop=new-shop.myshopify.com` (first time) 2. Read `tenants:index` from KV	`new-shop.myshopify.com` appears in the index array	[ ]	
UAT-MT-03	Tenant identity from query param	1. `GET /config?shop=shop-a.myshopify.com`	Config for shop-a returned	[ ]	
UAT-MT-04	Tenant identity from webhook header	1. `POST /webhooks/customer-update` with `X-Shopify-Shop-Domain: shop-a.myshopify.com`	Webhook processed using shop-a's config	[ ]	
UAT-MT-05	Missing tenant ID returns 400	1. `POST /config` without `?shop=` param (in multi-tenant mode)	`400 Missing shop parameter`	[ ]	
UAT-MT-06	Cron iterates all tenants	1. Register 2+ tenants 2. Trigger scheduled handler	Sync runs for each tenant in `tenants:index`; per-tenant errors isolated	[ ]	
UAT-MT-07	Legacy fallback (private plan)	1. Deploy with no `tenants:index` key 2. `GET /config` (no shop param)	Falls back to `crm_config` key	[ ]	

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-MT-08	Region-based tenant group	1. `POST /config?shop=us-hx.myshopify.com` 2. Read `tenants:index`	Entry has `region: "us"` (auto-inferred from prefix)	[ ]	
UAT-MT-09	Tenant registry structured entries	1. Register 2 tenants 2. `GET /admin/tenants` with bearer	Returns `TenantEntry[]` with `shop`, `region`, `registered_at`	[ ]	
UAT-MT-10	Tenant registry region filter	1. Register US + CA tenants 2. `GET /admin/tenants?region=us`	Returns only US-region tenants	[ ]	
UAT-MT-11	Per-tenant admin key	1. Set `admin_key` in tenant config 2. Use tenant key on admin endpoint	Tenant key accepted; platform key also still works	[ ]	
UAT-MT-12	Platform config CRUD	1. `GET /platform/config` with bearer 2. `POST /platform/config` with bearer	Platform config read/written at `platform:config` KV key	[ ]	
UAT-MT-13	Region provisioning	1. `POST /admin/provision-region?shop=us-hx.myshopify.com` with bearer	Xano tables created/verified; tag_table_ids cached in tenant-scoped KV	[ ]	
UAT-MT-14	Extension ?shop= param	1. Build extension bundle 2. Inspect `index.js`	All fetch calls include `?shop=` parameter (>= 4 references)	[ ]	
UAT-MT-15	KV key isolation	1. Trigger sync for shop A 2. Check KV keys	All KV keys use `tenant:{shop}:` prefix (tag_table_ids, webflow_tags_collection_id, sync:customers:last_run)	[ ]	
UAT-MT-16	OAuth state isolation	1. Start Webflow OAuth for two shops simultaneously 2. Complete one callback	Each flow has its own UUID-keyed state; no collision	[ ]	

7.12 UCP / Commerce Protocol Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-UCP-01	UCP Discovery Manifest	`GET /.well-known/ucp?shop=x`	200 with `protocol_version`, `capabilities`, `endpoints`, `authentication`, `Cache-Control: max-age=3600`	[ ]	
UAT-UCP-02	A2A Agent Card	`GET /.well-known/agent-card.json?shop=x`	200 with `name`, `version`, `skills` (>= 5), `authentication`	[ ]	
UAT-UCP-03	Commerce Identity (auth)	`GET /commerce/identity` with valid JWT	200 with `providers`, `consent_status` (incl. a2a, ap2), `tags`	[ ]	
UAT-UCP-04	Commerce Identity (unauth)	`GET /commerce/identity` without JWT	401	[ ]	
UAT-UCP-05	Product Catalog	`GET /commerce/products?shop=x`	200 with `products[]` and `pageInfo`	[ ]	
UAT-UCP-06	Product search	`GET /commerce/products?shop=x&query=test&first=5`	200 with filtered results	[ ]	
UAT-UCP-07	Orders (unauth)	`GET /commerce/orders?shop=x`	401	[ ]	
UAT-UCP-08	Checkout create (unauth)	`POST /commerce/checkout` without JWT	401	[ ]	
UAT-UCP-09	Checkout create (empty items)	`POST /commerce/checkout` with `line_items: []`	400	[ ]	

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-UCP-10	Mandate create (unauth)	`POST /commerce/mandates` without JWT	401	[ ]	
UAT-UCP-11	Mandate AP2 consent gate	`POST /commerce/mandates` with JWT but `consent_ap2 = false`	403 "AP2 agent payments consent required"	[ ]	
UAT-UCP-12	CORS on commerce routes	`OPTIONS /commerce/*`	`Access-Control-Allow-Methods` includes POST	[ ]	
UAT-UCP-13	Embed feature flag (enabled)	`GET /embed/footer?shop=x` (footer in `embeds_enabled`)	200	[ ]	
UAT-UCP-14	Embed feature flag (disabled)	`GET /embed/footer?shop=x` (footer removed from `embeds_enabled`)	403	[ ]	
~~UAT-UCP-15~~	~~Cart embed renders~~	Removed (v1.7) — cart handled by Shopify Web Components	N/A	N/A	
UAT-UCP-16	Checkout API field in manifest	`GET /.well-known/ucp?shop=x`	`checkout_api` = `storefront_cart` or `admin_draft_order`	[ ]	
UAT-UCP-17	A2A/AP2 capabilities in manifest	Enable a2a+ap2 consent, then `GET /.well-known/ucp`	Capabilities include `a2a_agent_access`, `ap2_agent_payments`	[ ]	
UAT-UCP-18	Storefront token auto-provisioned	Open Shopify app, check Settings	`shopify_storefront_token` field populated	[ ]	

7.13 Adobe AEP Integration Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOT
UAT-ADOBE-01	Adobe config fields in tenant config	1. `POST /config?shop=x` with Adobe credentials 2. `GET /config?shop=x`	Adobe fields saved and returned (secrets masked)	[ ]	
UAT-ADOBE-02	Adobe push on customer update	1. Enable `adobe_aep_enabled` in config 2. `POST /webhooks/customer-update` with customer data	AEP streaming endpoint called; sync status written to `user_extras`	[ ]	
UAT-ADOBE-03	SHA-256 PII hashing	1. Push customer with email `test@example.com` 2. Check AEP payload	Email hash matches `sha256(test@example.com)`, raw email never sent to AEP	[ ]	
UAT-ADOBE-04	Adobe IMS token refresh	1. Clear cached token 2. Trigger Adobe push	New token obtained via `client_credentials` grant; cached in KV	[ ]	
UAT-ADOBE-05	Adobe disabled (no-op)	1. Set `adobe_aep_enabled: false` 2. `POST /webhooks/customer-update`	No AEP call made; webhook completes normally	[ ]	
UAT-ADOBE-06	Xano schema setup	1. `POST /admin/adobe-schema?shop=x` with valid bearer	Adobe fields created on `user_extras` table; `adobe_sync_log` table verified	[ ]	
UAT-ADOBE-07	Webflow Adobe fields	1. `POST /admin/webflow-ensure-fields?shop=x`	5 Adobe fields (ECID, email hash, sync status, last synced, identity graph ID) present on Customers collection	[ ]	
UAT-ADOBE-08	Adobe fields in Webflow sync	1. Sync user with Adobe data in `user_extras` 2. Check Webflow CMS item	Adobe fields populated in Webflow CMS customer item	[ ]	

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOT
UAT-ADOBE-09	XDM consent mapping from tags	1. Add tags `newsletter_subscribed`, `accepts_marketing`, `rejects_ccpa` 2. Check AEP payload	`consents.marketing.email.val: "y"`, `consents.adID.val: "n"`, `consents.marketing.preferred: "in"`	[]	
UAT-ADOBE-10	Form bridge → AEP direct push	1. Submit form with `data-crm-form="newsletter"` 2. Check AEP events	`subscriptionEvent` with `syncSource: "form_bridge:newsletter"` appears in AEP	[]	
UAT-ADOBE-11	Product upsell → AEP	1. `POST /webhooks/upsell` with products + email 2. Check AEP events	`commerce.productListAdds` event with `productListItems` array in AEP	[]	
UAT-ADOBE-12	Upsell auto-tags user	1. `POST /webhooks/upsell` with `upsell_source: "checkout"` 2. Check Xano user	Tags `upsell_checkout` + `upsell_2026-05-17` added to user	[]	
UAT-ADOBE-13	Upsell → Shopify tags	1. `POST /webhooks/upsell` for user with Shopify GID 2. Check Shopify customer	Upsell tags synced to Shopify customer	[]	
UAT-ADOBE-14	Upsell → GA4 event	1. `POST /webhooks/upsell` 2. Check GA4 DebugView	`crm_upsell` event with `upsell_source`, `product_count`, `total_value` params	[]	

7.13 Pricing & Extension UI Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-PRICE-01	Plan tab renders	1. Open Webflow extension 2. Click "Plan" tab	Three pricing cards displayed: Shared ($69), Private ($325), Enterprise (Custom)	[ ]	
UAT-PRICE-02	Check subscription status	1. Click "Check Subscription Status"	Active features listed based on tenant config query	[ ]	
UAT-PRICE-03	Enterprise contact link	1. Click "Contact Sales" on Enterprise card	Opens mailto to `ysl@ysl150.com`	[ ]	
UAT-PRICE-04	Adobe config in extension	1. Click "Settings" tab 2. Scroll to Adobe section	Adobe AEP toggle, IMS Org ID, Client ID, Client Secret, Dataset ID, Sandbox fields visible	[ ]	
UAT-PRICE-05	Integrations upgrade note	1. View Settings tab	"Salesforce, HubSpot, Klaviyo, Attentive, Braze — upgrade required" note displayed	[ ]	

7.14 Token Lifecycle Tests

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	N
UAT-TKN-01	OAuth install issues expiring token	1. `GET /admin/shopify-install?shop=hx-stage.myshopify.com` 2. Complete OAuth flow	Token exchange includes `expiring=1`; KV stores access token, refresh token, and `shopify_token_expires_at`	[ ]	
UAT-TKN-02	Token type display	1. Open `/settings` 2. Check Shopify section	Token Type shows `OAuth (auto-refresh)` when refresh token present; Refresh Token shows `Present`	[ ]	
UAT-TKN-03	Manual force-refresh	1. `POST /admin/shopify-refresh` with bearer auth	Returns `{ refreshed: true, tokenPrefix: "...", apiResult: "OK" }`; new token saved to KV	[ ]	
UAT-TKN-04	Force-refresh unauthed	1. `POST /admin/shopify-refresh` without auth header	`401 Unauthorized`	[ ]	
UAT-TKN-05	API diagnostic endpoint	1. `GET /admin/shopify-test` with bearer auth	Returns token prefix, length, domain, and API call result	[ ]	
UAT-TKN-06	Cron auto-refresh	1. Set `shopify_token_expires_at` to 3 minutes from now in KV 2. Trigger scheduled handler	`refreshShopifyTokenIfNeeded()` fires (within 5-min buffer); new token and expiry saved to KV	[ ]	
UAT-TKN-07	Cron skips if not near expiry	1. Set `shopify_token_expires_at` to 2 hours from now 2. Trigger scheduled handler	`refreshShopifyTokenIfNeeded()` returns early; no token exchange call	[ ]	
UAT-TKN-08	Multi-tenant token isolation	1. Register 2 tenants with different tokens 2. Force-refresh tenant A	Tenant A gets new token; tenant B's token unchanged	[ ]	

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	N
UAT-TKN-09	App loader session sync	1. Open Shopify app (triggers `app.tsx` loader) 2. Check CRM worker KV	`shopify_admin_token` updated with fresh `session.accessToken`	[ ]	
UAT-TKN-10	Storefront token provisioned	1. Open Shopify app (triggers loader) 2. Check KV	`shopify_storefront_token` populated via `storefrontAccessTokenCreate`	[ ]	
UAT-TKN-11	Expired token returns 403	1. Set `shopify_admin_token` to a known-expired token 2. `GET /commerce/products?shop=x`	API call fails with 401/403; error logged	[ ]	
UAT-TKN-12	Missing refresh token skips refresh	1. Remove `shopify_refresh_token` from KV 2. Trigger cron	`refreshShopifyTokenIfNeeded()` returns early; no error thrown	[ ]	
UAT-TKN-13	Embed snippet format	1. Open Shopify app → Embed Codes tab	All snippets use fetch+innerHTML pattern with `var W=` and `var S=` (no bare long URLs)	[ ]	
UAT-TKN-14	Embed dual-format (HTML)	1. `fetch('/embed/account?shop=x')`	Response Content-Type: `text/html`; contains styles, markup, and `<script>` tags	[ ]	
UAT-TKN-15	Embed dual-format (JS)	1. `<script src="/embed/account?shop=x">` (browser sends `Sec-Fetch-Dest: script`)	Response Content-Type: `text/javascript`; self-injecting JS loader	[ ]	
UAT-TKN-16	Standalone Sign In fallback	1. Open `/embed/dashboard` directly in browser 2. Click Sign In	Redirects to `/auth/google/login?return_to=...` (fallback, no footer embed needed)	[ ]	

7.15 Shopify Admin App Dashboard Tests (FR-APP)

Automated regression coverage: `tests/shopify-app.test.sh` (`npm run test:app`).

ID	TEST CASE	STEPS	EXPECTED RESULT	PASS/FAIL	NOTES
UAT-APP-01	Dashboard renders	1. Open app from Shopify admin (Apps → CRM Sync)	Native `s-page` "CRM Sync" with Config / Embeds / Settings `s-button` tab strip; Config active by default	[ ]	
UAT-APP-02	Tab switch via URL	1. Click **Embeds** tab	URL gains `?tab=Embeds`; content swaps to embed snippets; **Settings** likewise → `?tab=Settings`	[ ]	
UAT-APP-03	Tabs work without client state	1. Load `/app?tab=Settings` directly (or with JS disabled)	Settings tab renders server-side — does not depend on client-only `useState` / `onClick`	[ ]	
UAT-APP-04	Auth params preserved	1. Switch between tabs 2. Inspect URL	`shop`, `host`, `embedded`, `id_token` params retained across the switch (App Bridge stays authenticated)	[ ]	
UAT-APP-05	Settings save	1. Settings tab 2. Fill a field 3. Save Changes	Config POSTed to worker `/config`; hidden `intent=save` ensures submit works pre-hydration	[ ]	
UAT-APP-06	Single-route consolidation	1. Request `/app/embeds` and `/app/settings`	Routes do not exist — all three tabs served from `/app`	[ ]	
UAT-APP-07	Unauthenticated bounce	1. `GET /app` (app worker `hx-crm-sync`) without a session	Shopify embedded auth bounce (`410` / `302`), never `404` / `500`	[ ]	

8. Known Limitations & Risks

#	ITEM	IMPACT	MITIGATION
1	Cloudflare Workers 50-subrequest limit per invocation	Init tag system split into 3 steps; cron pages at 50 customers	Batch operations stay under limit
2	Xano Meta API schema quirks (POST /field returns 404)	Schema updates require PUT with full schema array	Documented in INTERNAL-SETUP.md
3	GA4 Measurement Protocol events are server-side	No browser session stitching without GTM	`user_id` matching bridges server + client events
4	Webflow CMS API rate limits	High-volume syncs may be throttled	Cron pages at 100 users per sync
5	Single-file worker architecture (~9,400 lines, 78 routes)	Maintenance complexity	Acceptable for current scope; refactor if adding major features
6	KV eventual consistency	Config changes may take seconds to propagate	Edge-level consistency acceptable for config updates
7	Shopify expiring tokens require proactive refresh (mandatory since April 2026)	Access tokens expire on a Shopify-controlled TTL; non-expiring tokens return 403	Auto-refreshed via `*/15 * * * *` cron with 5-min buffer; `POST /admin/shopify-refresh` for manual force-refresh; tracked via `shopify_token_expires_at` in KV config
8	POST /config requires auth header after security hardening	Webflow Designer Extension updated to send `Authorization: Bearer`	Extension uses `shopifyAppSecret` as bearer token
9	Adobe AEP streaming ingestion is eventual	AEP may take minutes to reflect streamed events in profiles	Sync status tracked in `user_extras`; audit log in `adobe_sync_log`

#	ITEM	IMPACT	MITIGATION
10	Adobe IMS token caching in KV	Cached tokens may survive worker restarts	TTL-based expiry ensures refresh; manual invalidation via KV delete
11	Cloudflare Access OTP email delivery	OTP emails may not arrive for some email providers	Verified with `ysl@ysl150.com`; add alternative IdP if needed
12	Multi-tenant cron scales linearly	Each tenant adds sync time to cron handler	Per-tenant timeout isolation; consider parallel execution at scale
13	Storefront Cart API requires storefront token	Channel attribution only works with Cart API, not Draft Orders	Auto-provisioned on app auth; Draft Order fallback when unavailable
14	SSE streaming not yet implemented (Task 6)	A2A Agent Card declares `streaming: false`	Agent interactions are request/response only; SSE planned for next sprint
15	AP2 mandates require explicit consent	Customers must enable AP2 in account preferences before agents can create mandates	403 returned with actionable message guiding user to enable consent

9. Compliance Matrix

9.1 GDPR Article Mapping

ARTICLE	REQUIREMENT	IMPLEMENTATION
Art. 6	Lawful basis for processing	Consent-based: explicit opt-in for each processing purpose
Art. 7	Conditions for consent	Granular toggles per type, freely withdrawable
Art. 12	Transparent information	UCP Dashboard shows all consent states and audit history
Art. 15	Right of access	`POST /gdpr/data-request` returns complete user data
Art. 17	Right to erasure	`POST /gdpr/customer-redact` anonymizes all PII
Art. 20	Right to data portability	`POST /gdpr/data-request` returns structured JSON
Art. 25	Data protection by design	Consent required before data processing; audit trail immutable
Art. 30	Records of processing	`consent_records` table with source, timestamp, IP
Art. 32	Security of processing	HMAC-SHA256 webhook verification, OAuth state CSRF protection, `POST /config` authentication, embed HTML secret stripping, Cloudflare Access Zero Trust, ADMIN_KEY bearer auth on admin endpoints
Art. 35	Data protection impact assessment	Adobe AEP integration uses SHA-256 hashed PII only; raw PII never transmitted to third-party CDP
Art. 33	Breach notification	Cloudflare incident response + KV audit trail

9.2 CCPA Compliance

REQUIREMENT	IMPLEMENTATION
Right to know	Data request endpoint
Right to delete	Customer redaction endpoint
Right to opt-out of sale	CCPA toggle in consent settings
Non-discrimination	No feature gating based on privacy choices

10. Release Criteria

10.1 Go/No-Go Checklist

#	CRITERION	REQUIRED	STATUS
1	All PRE-UAT checks pass	Yes	[ ]
2	All UAT-AUTH tests pass (14 tests)	Yes	[ ]
3	All UAT-CON tests pass (8 tests)	Yes	[ ]
4	All UAT-TAG tests pass (at least 5/7)	Yes	[ ]
5	All UAT-GDPR tests pass (8 tests)	Yes	[ ]
6	All UAT-SEC tests pass (20 tests)	Yes	[ ]
7	UAT-GA4 tests pass (at least 3/5)	Yes	[ ]
8	UAT-SYNC tests pass (at least 3/5)	Yes	[ ]
9	UAT-FORM tests pass (at least 3/5)	No — depends on GTM setup	[ ]
10	All UAT-MT tests pass (16 tests)	Yes	[ ]
11	UAT-ADOBE tests pass (at least 5/8)	Yes	[ ]
12	UAT-UCP tests pass (at least 12/18)	Yes	[ ]
13	UAT-PRICE tests pass (at least 3/5)	No — UI-only	[ ]
13	No critical or high-severity defects open	Yes	[ ]
14	DPO sign-off on consent architecture	Yes	[ ]
15	PMO sign-off on release scope	Yes	[ ]

10.2 Defect Severity Classification

SEVERITY	DEFINITION	RELEASE IMPACT
Critical	Data loss, security breach, PII exposure	Blocks release
High	Core flow broken (auth, consent, sync fails)	Blocks release
Medium	Feature degraded but workaround exists	Release with known issue
Low	Cosmetic, non-functional, minor UX	Release acceptable

11. Sign-Off

DPO Approval

I have reviewed the consent architecture, GDPR compliance controls, audit trail implementation, and data protection measures described in this specification. The system meets the requirements for lawful data processing under GDPR and CCPA.

Name	_____
Title	Data Protection Officer
Signature	_____
Date	_____
Conditions	_____

PMO Approval

I have reviewed the functional requirements, test plan, release criteria, and known limitations. The system is approved for UAT testing and subsequent production release.

Name	_____
Title	Project Management Office
Signature	_____
Date	_____
Conditions	_____

12. Versioning Model

Five distinct version axes, each with its own record:

AXIS	SCOPE	RECORDED IN	BUMPED BY
Spec / doc	this document	revision history (§ top)	manual (currently v1.13)
Worker deploy	the running worker	Cloudflare Version IDs	every `wrangler deploy`
Entitlement `state_version`	per subject	`entitlements` (190)	every entitlement write (drives the version-monotonic consent guard)
Signing-key version	offline-token signer	`entitlement_signing_keys` (192), `next→active→retired`	`POST /entitlement/signing-keys/rotate`
Admin-key rotation	the gateway credential	`admin_key:meta` (rotations/timestamps)	`POST /admin/rotate-key` (a manual dashboard secret edit does **not** record meta)

Rules: `state_version` is strictly monotonic per subject and is the consistency clock for consent across channels; signing-key and admin-key rotations are non-destructive (overlap windows); spec version is bumped in lockstep with material capability changes.

13. Agency → Client Deploy Handoff (Interactive Key Rotation)

The handoff from implementing agency to client infrastructure is a **security ceremony with a paper trail**, not a credential copy. Normative checklist: `AGENCY-HANDOFF.md` (published in the public spec repo). Summary of the binding rules:

FR-HANDOFF-01 — Re-mint, never transfer. Every credential in the stack is replaced on client infrastructure in dependency order (CI deploy token → dispatch PAT → worker shared secrets → ADMIN_KEY root + rotatable → tenant

tokens → Xano keys (master + scoped entitlement:read) → Shopify app secret → Webflow site tokens → entitlement signing keys). Handoff is complete when the credential inventory shows zero agency-fingerprinted credentials alive.

FR-HANDOFF-02 — Interactive Key Rotation. Rotation is executed by a named **Security Human** (dashboard or CLI, in person); the **Agent** (AI/automation) prepares the runbook, verifies old-dead/new-alive on every consumer surface, and writes the audit record. Agents never mint, hold, or transport production credentials. Audit entries record date, credential, action, executed_by, verified_by, reason, SHA-256[:8] fingerprints (never secrets), overlap window, and consumers updated.

FR-HANDOFF-03 — Rotation triggers. Handoff, 90-day cadence, personnel change, suspected incident, or credential scope change.

FR-HANDOFF-04 — Compliance artifacts. The handoff produces the DPO package: PII map (identity store, commerce customers, consent records, analytics user properties), processor chain for the DPA annex, subject-rights rails (HMAC-verified data_request/redact, version-monotonic consent), the post-handoff access matrix (humans AND agents), and the rotation audit trail as custody evidence.

FR-HANDOFF-05 — Security-Scaling Report. Quarterly (or per market launch): credential inventory diff with ages (flag >90d), ceremonies performed, tenant token census, 401-anomaly summary, consent state_version monotonicity spot-check, and tier-boundary re-isolation (re-run FR-HANDOFF-01 when crossing shared → private → enterprise).

The full ceremony procedure — the human-executes/agent-verifies flow diagram, the trust-boundary model, and the fingerprint-only audit-record schema — is documented in `KEY-MANAGEMENT-LIFECYCLE.md` **§9 (Interactive Key Ceremony)** and surfaced as a featured capability in `SECURITY-POSTURE.md`.

Relationship to existing controls: builds on §3.10.2 (self-service admin key rotation), the §12 versioning axes (admin-key meta + signing-key versions are the machine-readable half of the audit trail), and §5/§9 (security controls, compliance matrix).

Glossary — Globalized Commerce

Vocabulary for the agentic, cross-border settlement layer surfaced on the **Payments** operator screen (`/embed/payments` → tabs **Rails · Sockets · Mandates · History**). This layer routes an agentic purchase from market discovery to capture-to-bank while Shopify stays the system of record.

Markets & rails *(tab: Rails)*

TERM	DEFINITION
Market	A shopper country (ISO alpha-2). Each market resolves to a default rail + settlement currency.
Rail	The settlement pathway for a purchase: `ucp` / `mpp` (Shopify-native, payment-agnostic), `kakaopay`, `samsungpay` (wallet), `stripe_jp`.
Settlement currency	The currency a rail pays out in (e.g. KRW, JPY), independent of the store/presentment currency Shopify records.
Market lifecycle	Governance state of a market's bespoke settlement rail: **RATIFIED** (live — human-approved to take real money), **INCUBATING** (built but not yet activated; cannot settle), **OPEN** (no bespoke rail — settles natively via Shopify/ `ucp`), **BLOCKED** (sanctioned — never settable).
Ratification	The one-time human governance action that flips a market `incubating → ratified`. Enforced fail-closed at runtime (`assertMarketActive`) and pre-deploy (deploy guard). Distinct from a transaction's *capture* (see History).

Settlement sockets *(tab: Sockets)*

TERM	DEFINITION
Settlement socket	A pluggable Country/Bank connector that captures money to a bank — Stripe, Kakao Pay, a domestic PG (e.g. KG Inicis). The same "socket" abstraction used for data joins, applied to money.
Acquirer / PSP	The payment service provider behind a socket that moves funds to card networks / bank rails (Visa/MC; KFTC in Korea).
Merchant of Record (MoR)	The legal entity that owns the transaction: `platform_local` (the platform's in-market entity) or `platform_xborder` (the platform's cross-border Stripe-eligible entity, e.g. US/JP/UK). The wallet (Samsung/Kakao tokenization) is never the MoR.
Health	Whether a socket's acquirer is configured and reachable (credentials present + health check passes) — shown as *healthy* / *down*.

Mandates & agents *(tab: Mandates)*

TERM	DEFINITION
AP2 mandate	A cryptographically signed authorization (Agent Payments Protocol) permitting an AI agent to transact on a shopper's behalf within explicit limits.
Agent	The AI agent authorized under a mandate.
Cap	The spending ceiling (max amount) bound to a mandate.
Scope	The product types / categories a mandate may purchase.
Human-present vs autonomous	Whether the final payment requires on-device human approval (e.g. Samsung Pay biometric) or may be executed server-side by the agent under the mandate (e.g. Kakao Pay).

Settlement history *(tab: History)*

TERM	DEFINITION
Capture	The per-transaction money event: the Merchant of Record settles the authorized purchase onto a bank-connected rail.
Settlement state	The money lifecycle of a transaction: `captured`, `refunded`, `returned`, `failed`.
Binding (JWE + Auth Me)	The trust step that ties a wallet credential (delivered as a JWE token) to the same authenticated identity (Xano Auth Me) that holds the mandate — asserted before any capture.

> **Two "ratify"s, kept distinct.** *Market ratification* (Rails) is a once-per-market governance flip — "this market may settle." *Settlement capture* (History) is a per-transaction money event by the Merchant of Record. The system never uses the same word for both.

Document generated 2026-05-14, updated v1.8 2026-05-26. Reflects CRM Sync worker (~9,500 lines, 81 route handlers) deployed to `crm.story-story.ai`. Multi-tenant SaaS with UCP/A2A/AP2 protocol compliance, A2A/AP2 consent toggles, 4 embed endpoints (footer, compliance, account, dashboard), Shopify client credentials token provisioning, mandatory expiring token rotation, region-based tenant groups, per-tenant admin keys, self-service admin key rotation, Adobe AEP, and three-tier pricing. Product display and cart/checkout handled externally by Shopify Web Components + PIM grid worker (`cf-worker-webflow-sync`).
